

Politechnika Wrocławska

Wydział Informatyki i Telekomunikacji

Kierunek: IST

**ZESPOŁOWE PRZEDSIĘWZIĘCIE
INFORMATYCZNE**

AI Present Finder

Dawid Chudzicki

Marcin Dolatowski

Bartosz Gotowski

Szymon Kowaliński

Opiekun pracy

dr inż. Marcin Jodłowiec

WROCŁAW 2025

Spis treści

DOKUMENTACJA PROJEKTOWA	4
1 Wykaz symboli, oznaczeń i akronimów	4
2 Cel i zakres przedsięwzięcia	4
2.1 Wstęp	4
2.2 Cel główny	5
2.3 Zakres projektu	5
2.4 Cele biznesowe i techniczne	5
3 Słownik pojęć	6
4 Stan wiedzy w obszarze przedsięwzięcia	6
5 Założenia wstępne	7
5.1 Założenia technologiczne	7
5.2 Założenia projektowe i ograniczenia	7
6 Analiza wymagań i sposób pracy	7
6.1 Iteracja 1: POC (Proof of Concept)	8
6.2 Iteracja 2: Design	8
6.3 Iteracja 3: MVP	9
6.4 Iteracja 4: Deploy	9
6.5 Organizacja pracy zespołu	10
6.6 Przypadki użycia	10
7 Specyfikacja wymagań na produkt programowy	13
7.1 Wymagania funkcjonalne	13
7.2 Wymagania нефункционалне	15
8 Projekt produktu programowego	17
8.1 Architektura systemu	18
8.2 Poziom 1: Kontekst systemu	18
8.3 Poziom 2: Kontenery	18
8.4 Poziom 3: Komponenty wybranych mikroservisów	22
8.5 Zastosowane wzorce projektowe	29
9 Model danych i baza danych	29
9.1 Model konceptualny	29
9.2 Implementacja w DBMS	37
10 Modelowanie behavioralne	37
10.1 Diagram sekwencji	37
10.2 BPMN	37

11 Prototypowanie interfejsu	41
11.1 Krok 1: Strona główna (Landing Page)	41
11.2 Krok 2: Formularz wyszukiwania	41
11.3 Krok 3: Wywiad z chatbotem	42
11.4 Krok 4: Lista rekomendacji	43
11.5 Krok 5: Nawigacja i zarządzanie	44
11.6 Panel administracyjny	45
12 Implementacja	46
12.1 Algorytm Rerankingu z Weryfikacją (XAI)	46
12.2 Orkiestracja zdarzeń w architekturze CQRS	46
12.3 Strukturyzacja rozmowy przez Tool Calling	47
12.4 Inżynieria promptów (Prompt Engineering)	48
12.5 Równoległa Agregacja Ofert	50
13 Testy	50
13.1 Planowanie testów	50
13.2 Testy z użytkownikami	51
14 Wyniki i analiza badań	52
14.1 Efektywność czasowa	52
14.2 Jakość rekomendacji	52
14.3 Przykłady rerankingu AI (Explainable AI)	53
14.4 Feedback użytkowników	54
14.5 Stabilność	54
15 Podsumowanie projektu	54
16 Wnioski i kierunki dalszego rozwoju	55

DOKUMENTACJA PROJEKTOWA

1 Wykaz symboli, oznaczeń i akronimów

Skrót	Pełna nazwa	Opis
AI	Artificial Intelligence	Sztuczna inteligencja – technologia wykorzystująca algorytmy uczenia maszynowego.
API	Application Programming Interface	Interfejs programistyczny umożliwiający komunikację między systemami.
UI	User Interface	Interfejs użytkownika.
UX	User Experience	Doświadczenie użytkownika.
VPS	Virtual Private Server	Wirtualny serwer prywatny.
OSINT	Open Source Intelligence	Biały wywiad, pozyskiwanie informacji z ogólnodostępnych źródeł.
LLM	Large Language Model	Duży model językowy (np. GPT-4, Gemini).
SSE	Server-Sent Events	Technologia przesyłania aktualizacji z serwera do klienta.
CQRS	Command Query Responsibility Segregation	Wzorzec architektoniczny rozdzielający odczyt od zapisu.
MVP	Minimum Viable Product	Wersja produktu z minimalną liczbą funkcji wystarczającą do wdrożenia.
POC	Proof of Concept	Dowód koncepcji, prototyp mający na celu weryfikację wykonalności.
SPA	Single Page Application	Aplikacja internetowa działająca w obrębie jednej strony HTML.
DTO	Data Transfer Object	Obiekt służący do przesyłania danych między podsystemami.
XAI	Explainable Artificial Intelligence	Wyjaśnialna sztuczna inteligencja, której decyzje są zrozumiałe dla ludzi.

Tabela 1: Wykaz symboli, oznaczeń i akronimów.

2 Cel i zakres przedsięwzięcia

2.1 Wstęp

Celem projektu jest opracowanie aplikacji webowej do proponowania spersonalizowanych prezentów dla osób na podstawie podanych linków do ich mediów społecznościowych, a także podanych zainteresowań podczas wywiadu z chatbotem. Na podstawie danych system generuje przykładowe propozycje prezentów, aby następnie wyszukać je w zintegrowanych serwisach sklepowych i zwrócić linki do ofert użytkownikowi.

2.2 Cel główny

Stworzenie użytecznego, szybkiego i bezpiecznego narzędzia wspierającego proces znajdowania prezentu poprzez:

- automatyczne wydobycie i analizę cech osoby na podstawie mediów społecznościowych (OSINT),
- interaktywny chat z agentem AI w celu zebrania dodatkowych informacji,
- generowanie trafnych propozycji prezentów na podstawie uzyskanych danych,
- proponowanie konkretnych ofert produktów ze sklepów internetowych (agregacja).

2.3 Zakres projektu

Co wchodzi w zakres projektu:

- interfejs webowy do wprowadzania linków do publicznych profili społecznościowych i prowadzenia chatu z użytkownikiem,
- moduł parsowania i ekstrakcji jedynie publicznych, jawnych danych z podanych profili,
- generowanie propozycji prezentów dla wybranej osoby,
- propozycja ofert prezentów ze sklepów internetowych (OLX, Allegro, eBay, Amazon),
- filtry wyników (np. po cenie, kategorii).

Co nie wchodzi w zakres projektu:

- automatyczne dokonywanie zakupów ani płatności,
- zbieranie prywatnych danych wymagających logowania,
- generowanie treści graficznych produktów (korzystamy z gotowych zdjęć ofert).

2.4 Cele biznesowe i techniczne

Z biznesowego punktu widzenia projekt ma na celu skrócenie czasu potrzebnego na znalezienie prezentu z kilkunastu godzin do kilku minut oraz zwiększenie trafności upominków. Z technicznego punktu widzenia celem jest weryfikacja możliwości wykorzystania modeli LLM do sterowania procesem wyszukiwania i selekcji ofert (Agentic Commerce) oraz integracja wielu źródeł danych w czasie rzeczywistym.

3 Słownik pojęć

Pojęcie	Opis / Definicja
AI Present Finder	Nazwa projektu — aplikacja webowa wykorzystująca sztuczną inteligencję do proponowania spersonalizowanych prezentów.
Użytkownik	Osoba korzystająca z aplikacji w celu znalezienia pomysłu na prezent dla innej osoby.
Osoba obdarowywana	Osoba, której profil jest analizowany przez system w celu znalezienia prezentu.
Chatbot / Agent AI	Moduł oparty na LLM prowadzący rozmowę z użytkownikiem, zadający pytania o preferencje, okazję i budżet.
Profil społecznościowy	Publicznie dostępna strona użytkownika w mediach społecznościowych (np. Instagram, TikTok).
Ekstrakcja danych	Proces automatycznego pobierania i analizy publicznych informacji z profili.
Propozycja prezentu	Wygenerowany przez AI ogólny pomysł na prezent (np. "Aparat analogowy").
Propozycja oferty	Konkretny link do produktu w sklepie internetowym (np. aukcja na Allegro).
Filtrowanie wyników	Zawężanie listy rekomendacji według kryteriów takich jak cena czy kategoria.
Dane publiczne	Ogólnodostępne informacje w Internecie, niewymagające logowania ani specjalnych uprawnień do odczytu.

4 Stan wiedzy w obszarze przedsięwzięcia

Rynek cyfrowego wsparcia w obdarowywaniu można podzielić na dwa główne segmenty: rozwiązania B2B (Corporate Gifting) oraz B2C (Consumer Chatbots). Rozwiązania B2B (np. Giftpack.ai) oferują wysoką jakość dzięki analizie danych, ale są niedostępne dla klienta indywidualnego. Rozwiązania B2C (np. DreamGift) są łatwo dostępne, ale często opierają się na prostych chatbotach bez kontekstu, co prowadzi do niskiej trafności rekomendacji.

Poniższa tabela (Tabela 2) przedstawia porównanie AI Present Finder z istniejącymi rozwiązaniami konsumenckimi:

Rozwiązanie	Źródła danych	Interakcja	Integracja e-commerce
AI Present Finder	Media społecznościowe + chatbot	Szczegółowy wywiad (ok. 20 pytań)	OLX, Amazon, eBay, Allegro
DreamGift	Tylko rozmowa	Krótko (ok. 6 pytań)	Amazon
GiftAssistant	3 pola tekstowe	Brak konwersacji	Amazon
Giftruly	Formularz	Kilka pól wyboru	Amazon
IntelliGift	Pola tekstowe	Szerokie dane	Amazon

Tabela 2: Porównanie rozwiązań rynkowych

Głównym wyróżnikiem AI Present Finder jest połączenie analizy OSINT (rozwiązującej problem "zimnego startu") z lokalną agregacją ofert (OLX, Allegro), co pozwala na znajdowanie unikatowych prezentów niedostępnych w globalnych katalogach typu Amazon. Charakterystyka konkurencyjnych rozwiązań w Tabeli 2 pochodzi z własnej eksploatacji interfejsów tych produktów (stan na listopad 2025 roku).

5 Założenia wstępne

5.1 Założenia technologiczne

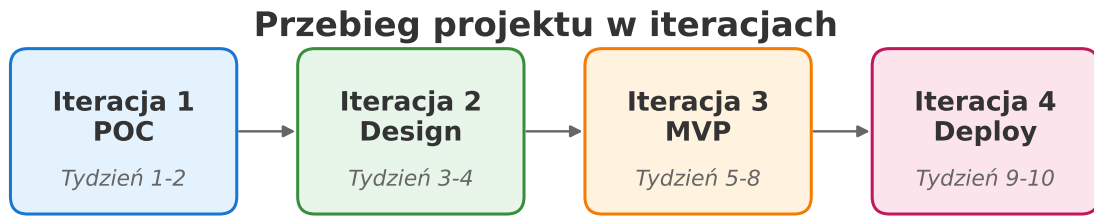
- **Frontend:** React (Single Page Application)
- **Backend:** NestJS (Node.js framework)
- **Baza danych:** PostgreSQL
- **Komunikacja:** RabbitMQ (kolejkowanie zadań), REST API, SSE
- **Integracje zewnętrzne:**
 - API mediów społecznościowych (poprzez proxy/scraping)
 - API e-commerce: OLX, Amazon, eBay, Allegro, Okazje (Sandbox / scraping)
 - OpenAI API / Google Gemini API (modele językowe)
- **Hosting:** VPS, obsługa HTTPS

5.2 Założenia projektowe i ograniczenia

- Przetwarzane są wyłącznie dane publicznie dostępne (OSINT).
- System nie obsługuje płatności ani finalizacji transakcji (przekierowanie do sklepu).
- Dane osobowe nie są wykorzystywane w celach marketingowych. Historia wyszukiwań i zapisane produkty są przechowywane wyłącznie dla zalogowanych użytkowników w celu zapewnienia ciągłości sesji.
- Projekt ma charakter edukacyjno-prototypowy (MVP).

6 Analiza wymagań i sposób pracy

Projekt był realizowany iteracyjnie w podejściu zbliżonym do Agile. Zespół korzystał z GitHub Issues jako backlogu produktu, dzieląc prace na kolejne iteracje (POC, Design, MVP, Deploy). Każdy issue reprezentował konkretne wymaganie lub zadanie, a kolumny statusu (Backlog / Todo / Done) odzwierciedlały postęp prac. Przebieg projektu przedstawiono na Rysunku 1.



Rysunek 1: Przebieg projektu w czterech iteracjach.

6.1 Iteracja 1: POC (Proof of Concept)

Celem pierwszej iteracji było sprawdzenie, czy najważniejsze ryzyka technologiczne są rozwiązywalne:

- przygotowanie **NestJS boilerplate** z obsługą zdarzeń (issue: "*NestJS boilerplate z zmockowanymi eventami*"),
- **Gift Search POC** – przetestowanie możliwości wyszukiwania prezentów w zewnętrznych serwisach,
- **POC scrapera Facebooka, Instagrama i Tiktoka** – weryfikacja, czy da się zbudować moduł OSINT,
- uruchomienie podstawowego **frontendu** ("set up basic frontend project"),
- przygotowanie pierwszych **diagramów**: C4, BPMN, projekt premise.

Rezultatem iteracji POC było potwierdzenie, że analiza profili, komunikacja zdarzeniowa i podstawowe wyszukiwanie produktów są możliwe w przyjętym stosie technologicznym. Na tym etapie powstały również pierwsze makiety interfejsu (Figma) oraz wstępny podział odpowiedzialności w zespole (backend, frontend, DevOps).

6.2 Iteracja 2: Design

Druga iteracja skupiła się na doprecyzowaniu architektury i modelu domeny:

- utworzenie **diagramów C4** (wersja ogólna i wersja mikroservisowa),
- przygotowanie **diagramu klas** oraz **diagramu BPMN** opisującego przepływ procesu,
- doprecyzowanie podziału na mikroservisy i kontraktów między nimi,
- konfiguracja środowiska developerskiego i CI.

Na koniec iteracji Design powstał spójny model architektoniczny, który został wykorzystany przy budowie MVP.

6.3 Iteracja 3: MVP

Trzecia iteracja miała na celu dostarczenie **działającego MVP** aplikacji:

- wyodrębnienie mikroserwisów: **gift-ideas-microservice**, **reranking-microservice**, **fetch-microservice**,
- integracja chatu z architekturą zdarzeniową ("EPIC: integrate chat POC with microservices setup"),
- dodanie widoków **frontendowych**: home, stalking, chat, rekomendacje,
- implementacja **historii czatu** oraz **historii rekomendacji prezentów**,
- dodanie **ulubionych produktów**, filtrowania i sortowania,
- pierwsza wersja **systemu rerankingu** i integracja z platformami e-commerce.

W trakcie iteracji MVP powstał też **landing page** oraz logotyp projektu. Celem biznesowym było udostępnienie wersji, która rozwiązuje podstawowy problem użytkownika: wygenerowanie sensownych propozycji prezentów w jednym miejscu.

Przykładowo, podczas jednego z wewnętrznych testów MVP okazało się, że standardowe wyszukiwanie produktów zwraca wiele duplikatów oraz ofert zupełnie niepasujących do profilu obdarowywanej osoby (np. części zamiennie zamiast gotowych prezentów). To konkretne doświadczenie doprowadziło do zaprojektowania dedykowanego modułu rerankingu opartego na AI, który odfiltrowuje szum i eksponuje najbardziej trafne propozycje.

6.4 Iteracja 4: Deploy

Ostatnia iteracja skupiła się na przygotowaniu systemu do beta-testów i wdrożenia produkcyjnego:

- dodanie **logowania i autoryzacji** (Google OAuth, uprawnienia, odświeżanie tokenów),
- rozbudowa **panelu użytkownika** oraz **panelu administratora**,
- dopracowanie **UX/UI** (animacje chatu, lepszy layout na desktopie, landing page),
- optymalizacje **rerankingu** (usuwanie duplikatów, pararelizacja oceniania, pętla regeneracji propozycji),
- **tłumaczenie** aplikacji na język polski,
- przygotowanie **deploymentu** (CI/CD, migracje bazy danych, health-checki).

W tej iteracji skoncentrowano się również na przygotowaniu aplikacji do testów z realnymi użytkownikami oraz zbieraniu feedbacku (gwiazdki, formularze opinii). Pojawiły się też mechanizmy refinementu (dopytywanie o lepsze prezenty na podstawie wcześniejszych wyborów) oraz pytanie o budżet przed rozpoczęciem wywiadu.

Na podstawie tak zebranych i iteracyjnie doprecyzowywanych wymagań przygotowano poniższą specyfikację produktu programowego.

6.5 Organizacja pracy zespołu

Zespół pracował w składzie czteroosobowym, gdzie każdy członek zespołu miał umiejętności pozwalające mu na pracę na dowolnym poziomie stacku technologicznego (tkz. generaliści):

- Dawid Chudzicki – Project Manager i Fullstack developer, odpowiedzialny za większość makroserwisu REST API, większość widoków frontendowych, moduł autentykacji (Google OAuth), doskonalenie flow AI i beta testy.
- Marcin Dolatowski – Fullstack developer, odpowiedzialny za integracje zewnętrzne (fetch-microservice, sklepy), pierwszą wersję scoringu oraz refaktoryzację i dopracowanie interfejsu użytkownika (landing page, część UI).
- Bartosz Gotowski – DevOps i infrastruktura (Coolify, Docker, CI/CD, migracje bazy danych), monitoring oraz dokumentacja techniczna (LaTeX, diagramy C4/BPMN). Oraz fullstack developer odpowiedzialny za stalking minicroservice. Twórca mockupów UI w figmie.
- Szymon Kowaliński – Fullstack developer, odpowiedzialny za zmiany architektoniczne, implementację SSE oraz główny odpowiedzialny za wywiad z użytkownikiem i reranking microservice. Poprawki i optymalizacje na styku frontend–backend, doskonalenie flow AI. Twórca plakatu i fiszki.

Postęp prac był regularnie raportowany prowadzącemu w formie statusów (co kto aktualnie robi, jaki jest plan na sprint i jakie pojawiły się ryzyka), co ułatwiało korygowanie zakresu MVP oraz planowanie kolejnych iteracji (m.in. sprint poświęcony testom i optymalizacji aplikacji).

6.6 Przypadki użycia

Na podstawie zebranych wymagań zdefiniowano najistotniejsze przypadki użycia (Use Case), m.in.:

- U1: Użytkownik podaje linki do profili społecznościowych osoby obdarowywanej.
- U2: Użytkownik przeprowadza wywiad z chatbotem (okazja, budżet, preferencje).
- U3: System analizuje dane (OSINT + wywiad) i generuje listę pomysłów na prezenty.
- U4: System wyszukuje i prezentuje oferty produktów z wielu sklepów.
- U5: Użytkownik filtruje i sortuje wyniki, dodaje produkty do ulubionych i historii.
- U6: Administrator przegląda statystyki i zarządza użytkownikami (panel admina).

Graficzna reprezentacja tych przypadków została przygotowana w postaci diagramu przypadków użycia UML (Rysunek 2), gdzie aktorami są m.in. *Użytkownik*, *Administrator* oraz zewnętrzne systemy (sklepy, media społecznościowe).

Zidentyfikowane przypadki użycia oraz wymagania zebrane w trakcie iteracji stanowią fundament dla formalnej specyfikacji wymagań, którą przedstawiono w kolejnym rozdziale.

Szczegółowy opis przypadków użycia

Powyższa lista U1–U6 przedstawia skrótowo najważniejsze funkcje systemu z perspektywy biznesowej. Poniższy opis rozwija je w postaci scenariuszy U1–U7 (np. U1–U4 składają się na jeden główny przepływ „Szukanie prezentu”, a U5 został rozbity na osobne przypadki: ulubione, filtrowanie i korzystanie z historii). Opis ma charakter biznesowy i koncentruje się na intencjach użytkownika, danych wejściowych/wyjściowych oraz alternatywnych ścieżkach; diagram pozostaje bez zmian.

U1. Szukanie prezentu (główny przepływ).

- **Warunki wstępne:** Użytkownik jest zalogowany. Ma opcjonalnie linki do profili społecznościowych oraz budżet.
- **Scenariusz główny:** Użytkownik podaje linki i budżet, następnie przechodzi krótki wywiad z chatbotem. Na podstawie profili i odpowiedzi system buduje profil odbiorcy, generuje pomysły na prezenty i zwraca listę realnych ofert z wielu sklepów. Oferty pojawiają się stopniowo, bez czekania na wszystkie źródła.
- **Przebiegi alternatywne:** Brak linków — system opiera się wyłącznie na wywiadzie. Część sklepów niedostępna — lista jest niepełna, ale nadal rośnie w miarę dostępności. Użytkownik pomija część pytań — system działa na mniejszym kontekście, później można doprecyzować. Użytkownik nie jest zalogowany — przed rozpoczęciem wyszukiwania zostaje przekierowany do przypadku U3 (logowanie przez Google).
- **Warunki końcowe:** Użytkownik widzi listę ofert dopasowanych do odbiorcy oraz ma zapisaną sesję do wglądu w historii.

U2. Dodanie produktu do ulubionych / historii.

- **Warunki wstępne:** Użytkownik jest zalogowany i ogląda wygenerowane oferty.
- **Scenariusz główny:** Użytkownik oznacza wybrane oferty jako ulubione; może je później zobaczyć w zakładce „Zapisane” lub w historii danej sesji.
- **Przebiegi alternatywne:** Próba zapisania oferty spoza własnej sesji jest blokowana; powtórne zapisanie tej samej oferty nie zmienia wyniku.
- **Warunki końcowe:** Lista ulubionych jest dostępna do ponownego otwarcia oferty lub kontynuacji rozmowy.

U3. Logowanie przez Google.

- **Warunki wstępne:** Użytkownik posiada konto Google.
- **Scenariusz główny:** Użytkownik przechodzi proces logowania Google, po którym uzyskuje dostęp do funkcji wymagających konta (historia, ulubione, feedback).
- **Przebiegi alternatywne:** Rezygnacja w trakcie logowania lub błąd uwierzytelnienia — brak dostępu do funkcji konta; użytkownik może spróbować ponownie.
- **Warunki końcowe:** Sesja użytkownika jest aktywna i pozwala korzystać z funkcji powiązanych z kontem.

U4. Wystaw opinię (feedback).

- **Warunki wstępne:** Użytkownik jest zalogowany i ma zakończoną sesję wyszukiwania.
- **Scenariusz główny:** Użytkownik ocenia jakość rekomendacji (gwiazdki, komentarz), może załączyć zdjęcia lub wskazać konkretną ofertę. Opinie są widoczne w panelu administratora i mogą wspierać dalsze ulepszenia.
- **Przebiegi alternatywne:** Brak wskazania konkretnej oferty — feedback traktowany jako ogólny. Brak komentarza — zapisywana jest sama ocena.
- **Warunki końcowe:** Opinia zostaje zapisana i może być analizowana przez zespół/administratora.

U5. Doprecyzuj wyszukiwanie (refinement).

- **Warunki wstępne:** Użytkownik ma wygenerowaną listę ofert w danej sesji.
- **Scenariusz główny:** Użytkownik wybiera kilka propozycji, które mu się podobają lub nie, i uruchamia kolejną rundę wyszukiwania. System proponuje nowy zestaw ofert na bazie preferencji z poprzedniej rundy.
- **Przebiegi alternatywne:** Brak zaznaczonych ofert — doprecyzowanie nie startuje. Niedostępne źródła — kolejna runda zawiera wyniki z dostępnych sklepów.
- **Warunki końcowe:** Użytkownik otrzymuje dodatkowe, lepiej dopasowane oferty w tej samej sesji.

U6. Filtruj / sortuj wyniki.

- **Warunki wstępne:** Są dostępne oferty dla danej sesji.
- **Scenariusz główny:** Użytkownik zawęży listę po cenie, dostawcy, ulubionych lub rundzie wyszukiwania i sortuje wyniki, by szybciej znaleźć właściwy prezent.
- **Przebiegi alternatywne:** Jeśli część źródeł nie zwróciła wyników, filtry działają na dostępnych ofertach; system może sygnalizować ograniczoną liczbę wyników.
- **Warunki końcowe:** Użytkownik widzi zawężoną, posortowaną listę i może podjąć decyzję.

U7. Panel administracyjny (feedbacki i statystyki).

- **Warunki wstępne:** Użytkownik posiada rolę administratora.
- **Scenariusz główny:** Administrator przegląda zgromadzone opinie oraz podstawowe statystyki (liczbę opinii, liczbę sesji z opiniami, średnią ocenę), aby monitorować jakość rekomendacji oczami użytkowników.
- **Przebiegi alternatywne:** Brak danych (np. brak feedbacków) — panel wyświetla puste zestawienia, ale pozostaje dostępny.
- **Warunki końcowe:** Administrator ma wgląd w kondycję produktu z perspektywy użytkowników; w ramach MVP panel pełni wyłącznie funkcję raportową (bez bezpośrednich zmian w danych użytkowników).

7 Specyfikacja wymagań na produkt programowy

Poniższa specyfikacja formalizuje wymagania zidentyfikowane w trakcie kolejnych iteracji i pracy z backlogiem, tak aby można je było jednoznacznie odwzorować w architekturze i implementacji.

7.1 Wymagania funkcjonalne

Poniższe wymagania opisują *co* system ma robić z perspektywy użytkownika. Dla każdego ujęto skrócony opis oraz kryteria akceptacji, które pozwalają weryfikować spełnienie wymagania podczas testów.

F1. Zebranie kontekstu o osobie obdarowywanej.

- **Opis:** System powinien umożliwiać użytkownikowi szybkie przekazanie podstawowych informacji o osobie obdarowywanej (linki do profili społecznościowych, okazja, przybliżony budżet), tak aby dalsze etapy mogły być maksymalnie zautomatyzowane.
- **Kryteria akceptacji:**
 - Przeciętny użytkownik jest w stanie wypełnić formularz startowy w ciągu jednej wizyty, bez potrzeby sięgania po dokumentację techniczną.
 - System akceptuje zarówno scenariusz z podaniem linków do profili, jak i scenariusz, w którym użytkownik opiera się wyłącznie na odpowiedziach w wywiadzie.

F2. Wykorzystanie danych publicznych (OSINT).

- **Opis:** System powinien wykorzystywać publicznie dostępne informacje z profili społecznościowych do lepszego zrozumienia zainteresowań i stylu życia obdarowywanej osoby, bez sięgania po dane prywatne.
- **Kryteria akceptacji:**
 - Dla osób, dla których dostępne są publiczne profile, użytkownicy subiektywnie oceniają rekomendacje jako lepiej dopasowane niż w scenariuszu bez linków (na podstawie zebranych opinii).
 - Brak dostępu do profilu lub brak linków nie blokuje procesu znajdowania prezentu — użytkownik może nadal przejść pełny flow oparty wyłącznie na wywiadzie.

F3. Wywiad z użytkownikiem.

- **Opis:** System powinien przeprowadzić użytkownika przez krótki, zrozumiały wywiad dotyczący relacji z obdarowywanym, okazji i jego preferencji, tak aby nie wymagać od użytkownika formułowania technicznych zapytań.
- **Kryteria akceptacji:**

- Użytkownik otrzymuje serię pytań w naturalnym języku, z możliwością udzielania odpowiedzi tekstowych lub wybierania z sugerowanych opcji.
- Po zakończeniu wywiadu użytkownik nie musi ręcznie konfigurować dodatkowych parametrów — system przechodzi do wyszukiwania prezentów automatycznie.

F4. Ukryta analiza profilu odbiorcy.

- **Opis:** System powinien łączyć odpowiedzi z wywiadu i dane publiczne w wewnętrzny profil odbiorcy, używany do podejmowania decyzji przez moduły AI, bez konieczności prezentowania tego profilu użytkownikowi w formie technicznej.
- **Kryteria akceptacji:**
 - Dla wyraźnie różnych typów odbiorców (np. osoba techniczna vs. artystyczna) generowane listy prezentów różnią się zauważalnie co do charakteru propozycji.
 - Użytkownicy w badaniach jakościowych potrafią wskazać, które elementy profilu obdarowywanego zostały dobrze uchwycone przez system (na podstawie opisu rekomendacji), mimo że profil jako taki nie jest im wyświetlany.

F5. Generowanie listy potencjalnych prezentów.

- **Opis:** System powinien na podstawie zbudowanego profilu wewnętrznie wygenerować listę możliwych typów prezentów i wykorzystać ją wyłącznie jako etap pośredni przed pokazaniem gotowych ofert.
- **Kryteria akceptacji:**
 - Użytkownik końcowy widzi jedynie gotowe propozycje produktów; szczegóły wewnętrznej listy pomysłów nie są eksponowane w interfejsie.
 - Dla tego samego zestawu danych wejściowych lista produktów pozostaje spójna tematycznie (np. wszystkie dotyczą fotografii, gier, gotowania itp.).

F6. Znalezienie realnych ofert w sklepach.

- **Opis:** System powinien przekuć wygenerowane pomysły na konkretne, realne oferty z wielu niezależnych platform sprzedażowych, tak aby użytkownik mógł od razu przejść do zakupu.
- **Kryteria akceptacji:**
 - Dla typowego scenariusza (z wypełnionym formularzem i wywiadem) użytkownik otrzymuje co najmniej kilkadziesiąt propozycji produktów, pochodzących z co najmniej trzech różnych platform sprzedażowych.
 - Każda propozycja zawiera minimalny zestaw informacji biznesowych: nazwę, cenę, miniaturę/zdjęcie (jeśli dostępne) oraz link prowadzący do miejsca zakupu.

F7. Zawężanie listy i oznaczanie ulubionych.

- **Opis:** System powinien umożliwiać użytkownikowi zawężenie długiej listy prezentów do kilku najbardziej interesujących pozycji oraz ich oznaczanie jako ulubione, tak aby łatwo wrócić do nich później.
- **Kryteria akceptacji:**
 - Użytkownik może w intuicyjny sposób ograniczyć listę widocznych prezentów według kilku prostych kryteriów (np. poziomu ceny czy źródła) bez konieczności znajomości składni zapytań.
 - Użytkownik może oznaczyć wybrane produkty jako ulubione i znaleźć je później w jednym, dedykowanym miejscu w aplikacji.

F8. Historia sesji i możliwość powrotu.

- **Opis:** System powinien zapamiętywać sesje wyszukiwania, aby użytkownik mógł wrócić do poprzednich wyników, przemyśleć je na spokojnie lub kontynuować rozpoczęty wcześniej proces.
- **Kryteria akceptacji:**
 - Użytkownik widzi listę swoich sesji wyszukiwania z podstawowymi informacjami (nazwa, data, liczba znalezionych prezentów).
 - Użytkownik może otworzyć ponownie wybraną sesję i zobaczyć te same wyniki, bez konieczności ponownego wprowadzania danych.

F9. Opinie użytkowników i wgląd administracyjny.

- **Opis:** System powinien umożliwiać użytkownikom przekazywanie prostych opinii o jakości rekomendacji, a administratorowi — przeglądanie zanonimizowanych statystyk tych opinii.
- **Kryteria akceptacji:**
 - Użytkownik może po zakończonej sesji ocenić rekomendacje w prosty sposób (np. skala gwiazdkowa, krótki komentarz).
 - Administrator ma dostęp do zagregowanych danych (liczba opinii, liczba sesji z opiniami, średnia ocena), pozwalających ocenić jakość rozwiązania na poziomie całego systemu.

7.2 Wymagania niefunkcjonalne

Wymagania niefunkcjonalne określają *jak dobrze* system powinien realizować funkcje zdefiniowane powyżej. Dla każdego z nich zdefiniowano mierzalne kryteria akceptacji.

NF1. Dostępność i obsługiwane platformy.

- **Opis:** Aplikacja działa w przeglądarce bez konieczności instalacji dodatkowego oprogramowania, z responsywnym interfejsem dopasowanym do urządzeń desktopowych i mobilnych.
- **Kryteria akceptacji:**
 - System jest dostępny w najnowszej stabilnej wersji co najmniej jednej popularnej przeglądarki (np. Chrome) bez zauważalnych błędów blokujących; wsparcie pozostałych przeglądarek jest celem dalszego rozwoju.
 - Kluczowe scenariusze (U1–U2) są wykonalne zarówno na desktopie, jak i na typowym urządzeniu mobilnym (potwierdzone testami ręcznymi na przynajmniej jednym reprezentatywnym urządzeniu).
 - Wejście na stronę nie wymaga instalacji pluginów ani rozszerzeń poza standardowo dostępną przeglądarką.

NF2. Wydajność i czas odpowiedzi.

- **Opis:** System powinien w akceptowalnym dla użytkownika czasie reagować na działania, w szczególności podczas wywiadu i wyszukiwania prezentów, przy czym dopuszcza się, że pełne wyniki mogą być dostępne dopiero po kilku minutach.
- **Kryteria akceptacji:**
 - Po rozpoczęciu wywiadu lub wyszukiwania użytkownik w krótkim czasie widzi wyraźny sygnał, że system pracuje (zmiana widoku, komunikat, pasek postępu) — brak wrażenia, że aplikacja „zawiesiła się”.
 - W sytuacjach, gdy generowanie pierwszych sensownych rekomendacji może zająć kilka minut, interfejs informuje o tym użytkownika (np. komunikatem typu „to może potrwać kilka minut”) oraz prezentuje pasek postępu lub inny wskaźnik.
 - Pełne wyniki mogą pojawiać się stopniowo, ale użytkownik ma przez cały czas wgląd w postęp (procent, etapy) i może przerwać proces bez utraty już wyświetlonych propozycji.

NF3. Bezpieczeństwo i poufność danych.

- **Opis:** System musi chronić dane użytkowników oraz osoby obdarowywanej, ograniczając się do publicznych danych i zapewniając bezpieczną komunikację.
- **Kryteria akceptacji:**
 - Cała komunikacja pomiędzy przeglądarką a backendem odbywa się z użyciem protokołu HTTPS (brak mieszanej treści HTTP na stronach aplikacji).
 - System nie prosi o dane logowania do serwisów społecznościowych obdarowywanej osoby ani nie wykorzystuje prywatnych informacji wymagających logowania.
 - Użytkownik może w każdej chwili wylogować się; po wylogowaniu dostęp do historii, ulubionych i panelu admina jest zablokowany.

NF4. Skalowalność i odporność na obciążenie.

- **Opis:** Architektura ma umożliwiać skalowanie obciążonych komponentów (np. wyszukiwania ofert, rerankingu) bez wpływu na resztę systemu.
- **Kryteria akceptacji:**
 - Architektura oraz konfiguracja uruchomieniowa pozwalają na uruchomienie dodatkowych instancji mikroserwisów odpowiedzialnych za wyszukiwanie i reranking bez konieczności modyfikacji interfejsu użytkownika (skalowanie poziome jest możliwe na poziomie infrastruktury).
 - Awaria pojedynczej instancji serwisu odpowiedzialnego za pobieranie ofert nie blokuje pozostałych funkcji systemu (chat, historia, ulubione nadal działają, choć lista ofert może być niepełna).
 - System poprawnie obsługuje sytuację, w której jeden z dostawców ofert (np. konkretny sklep) przestaje odpowiadać — użytkownik nadal otrzymuje częściowe wyniki i czytelną informację o ograniczeniu (np. brak ofert z danego źródła).

NF5. Obsługa błędów i komunikaty dla użytkownika.

- **Opis:** W przypadku błędów zewnętrznych serwisów, problemów z siecią lub nietypowych danych wejściowych system powinien zachowywać się przewidywalnie i komunikować problem językiem zrozumiałym dla użytkownika.
- **Kryteria akceptacji:**
 - Dla typowych błędów (brak wyników, brak odpowiedzi z jednego źródła, niepoprawny link) użytkownik widzi czytelny komunikat w UI, a aplikacja nie zawiesza się.
 - W przypadku braku jakichkolwiek ofert system sugeruje możliwe działania (np. zmianę budżetu, inny profil, odświeżenie wyszukiwania), zamiast prezentować pustą stronę.
 - Błędy techniczne (np. wewnętrzne komunikaty serwera) nie są ujawniane bezpośrednio użytkownikowi końcowemu.

Powyższy zestaw wymagań funkcjonalnych i нефункциональных stał się podstawą do opracowania architektury systemu, opisanej szczegółowo w następnej sekcji.

8 Projekt produktu programowego

Zdefiniowane wymagania funkcjonalne i нефункциональные zostały odwzorowane w poniższej architekturze mikroserwisowej, obejmującej zarówno warstwę frontendową, jak i zestaw współpracujących usług backendowych.

8.1 Architektura systemu

System został zaprojektowany w architekturze mikroserwisowej, co zapewnia skalowalność i separację odpowiedzialności. Komunikacja między serwisami odbywa się asynchronicznie za pośrednictwem kolejki wiadomości RabbitMQ.

Główne komponenty systemu:

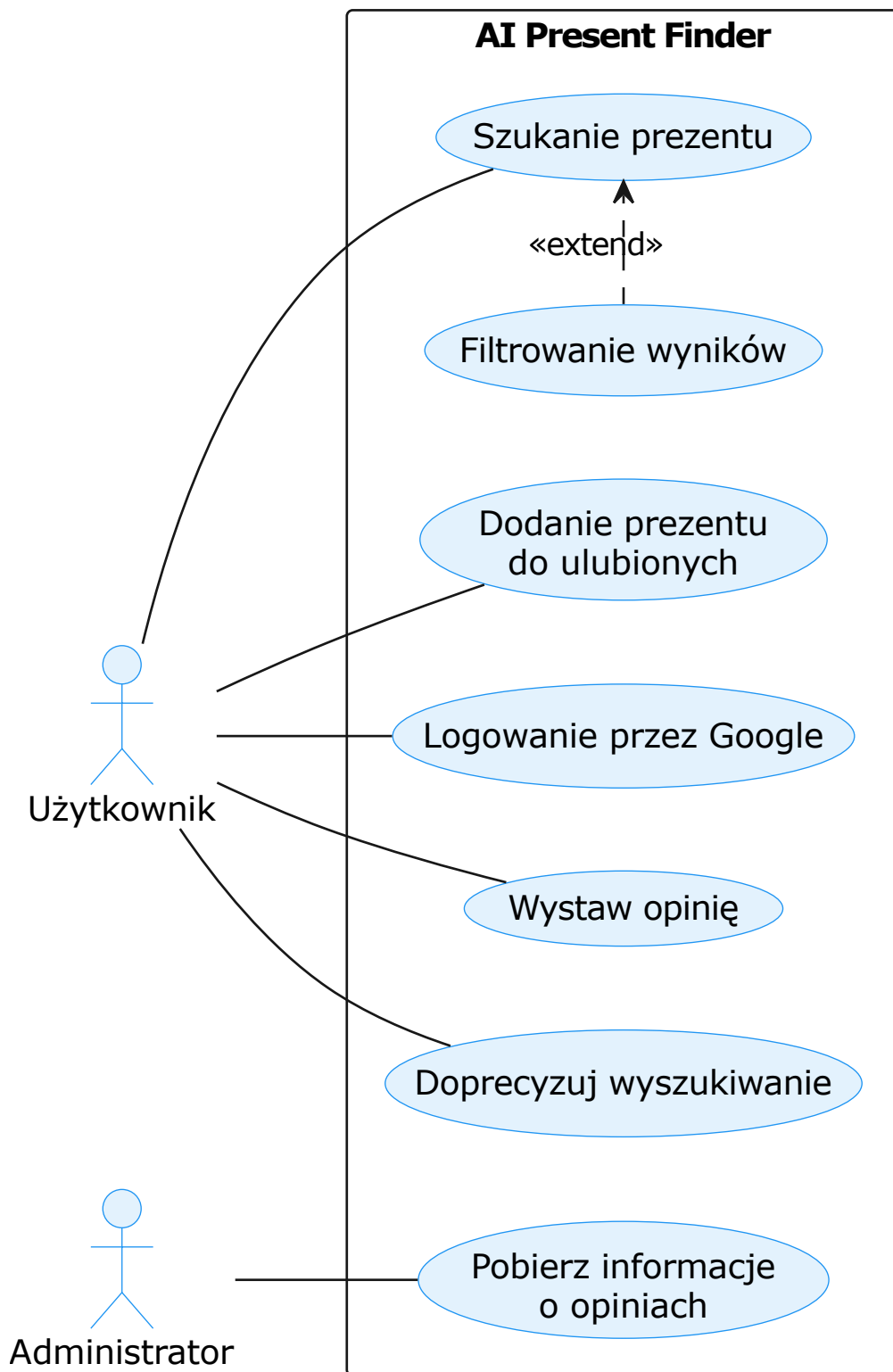
- **Frontend (React):** Interfejs użytkownika komunikujący się z backendem przez REST API oraz odbierający aktualizacje w czasie rzeczywistym przez SSE (Server-Sent Events).
- **RestAPI Macroservice (restapi-macroservice):** Brama wejściowa (API Gateway), obsługująca żądania HTTP i przekierowująca je do odpowiednich kolejek.
- **Chat Microservice (chat-microservice):** Odpowiada za prowadzenie wywiadu z użytkownikiem.
- **Stalking Microservice (stalking-microservice):** Realizuje proces OSINT (analiza profili społecznościowych).
- **Gift Ideas Microservice (gift-ideas-microservice):** Generuje abstrakcyjne pomysły na prezenty na podstawie zebranych danych.
- **Fetch Microservice (fetch-microservice):** Agreguje oferty z zewnętrznych platform (OLX, Allegro, Amazon, eBay, Okazje) przy użyciu dedykowanych handlerów per źródło. Działa w wielu instancjach równolegle.
- **Reranking Microservice (reranking-microservice):** Ocenia i filtruje znalezione oferty przy użyciu AI.

8.2 Poziom 1: Kontekst systemu

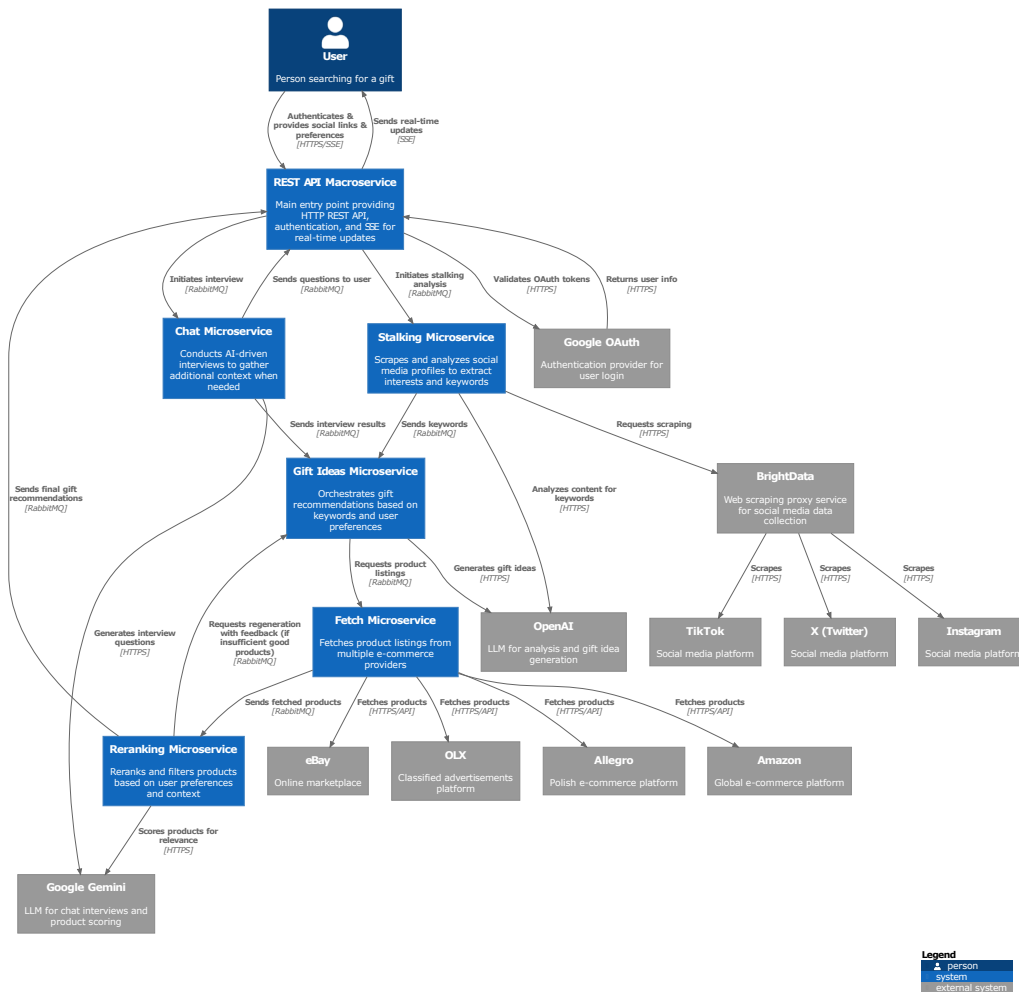
Na najwyższym poziomie abstrakcji (C4 Context) system AI Present Finder jest postrzegany jako pojedynczy blok współpracujący z aktorami zewnętrznymi: użytkownikiem szukającym prezentu, platformami e-commerce (OLX, Allegro, Amazon, eBay), mediami społecznościowymi (Instagram, TikTok, X) oraz dostawcami modeli LLM (OpenAI, Google). Zależności te przedstawiono na Rysunku 3.

8.3 Poziom 2: Kontenery

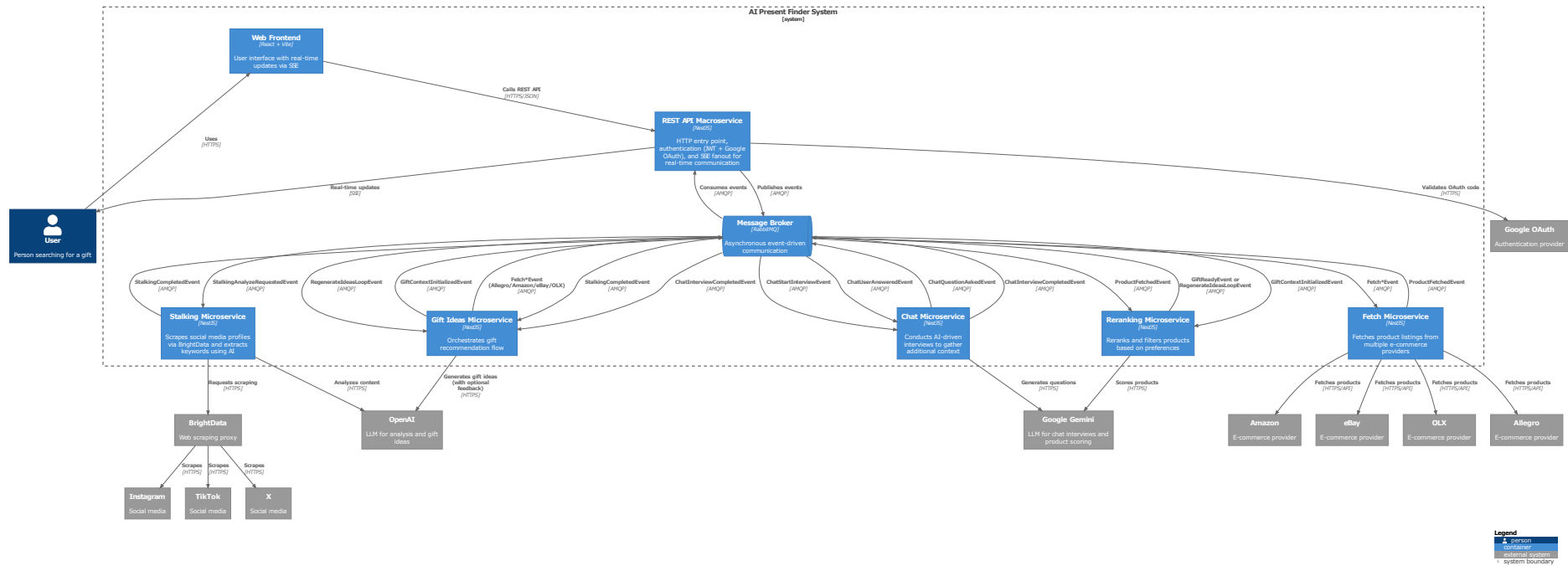
Po zejściu o jeden poziom niżej (C4 Container) widać podział systemu na odrębne jednostki wdrożeniowe: aplikację frontendową (React SPA), bramę API (RestAPI Macroservice), zestaw mikroserwisów backendowych, bazę danych PostgreSQL oraz broker wiadomości RabbitMQ. Każdy kontener może być niezależnie skalowany i wdrażany w środowisku Docker/Coolify. Architekturę kontenerów przedstawiono na Rysunku 4.



Rysunek 2: Diagram przypadków użycia systemu AI Present Finder.



Rysunek 3: Diagram kontekstowy systemu AI Present Finder (C4 Context).

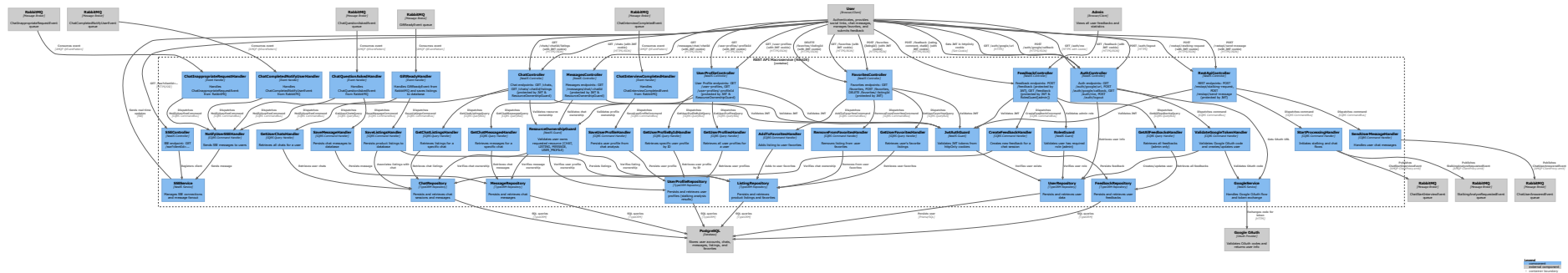


Rysunek 4: Diagram kontenerów systemu AI Present Finder (C4 Container).

8.4 Poziom 3: Komponenty wybranych mikroservisów

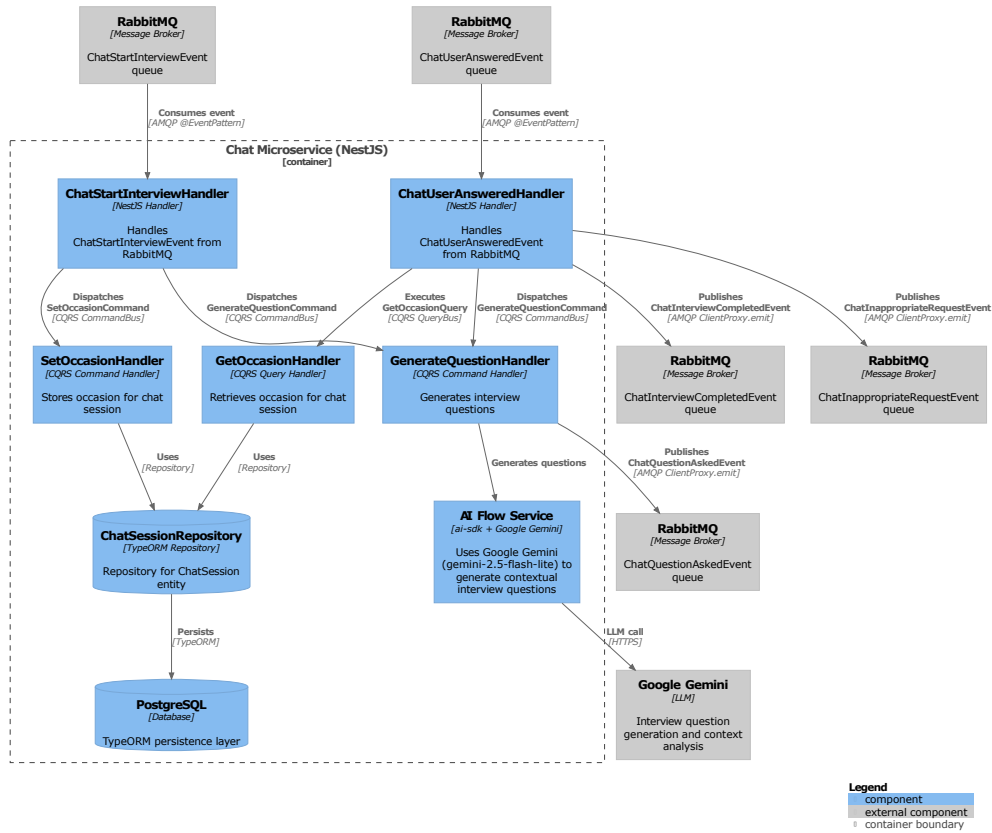
Na najniższym prezentowanym poziomie (C4 Component) każdy mikroservis jest rozbity na wewnętrzne moduły. Poniżej przedstawiono trzy najważniejsze serwisy.

RestAPI Macroservice. Brama wejściowa systemu (Rysunek 5) składa się z warstwy kontrolerów HTTP, handlerów komend CQRS oraz adapterów publikujących zdarzenia do kolejek RabbitMQ. Odpowiada również za obsługę SSE i autoryzację (Google OAuth).



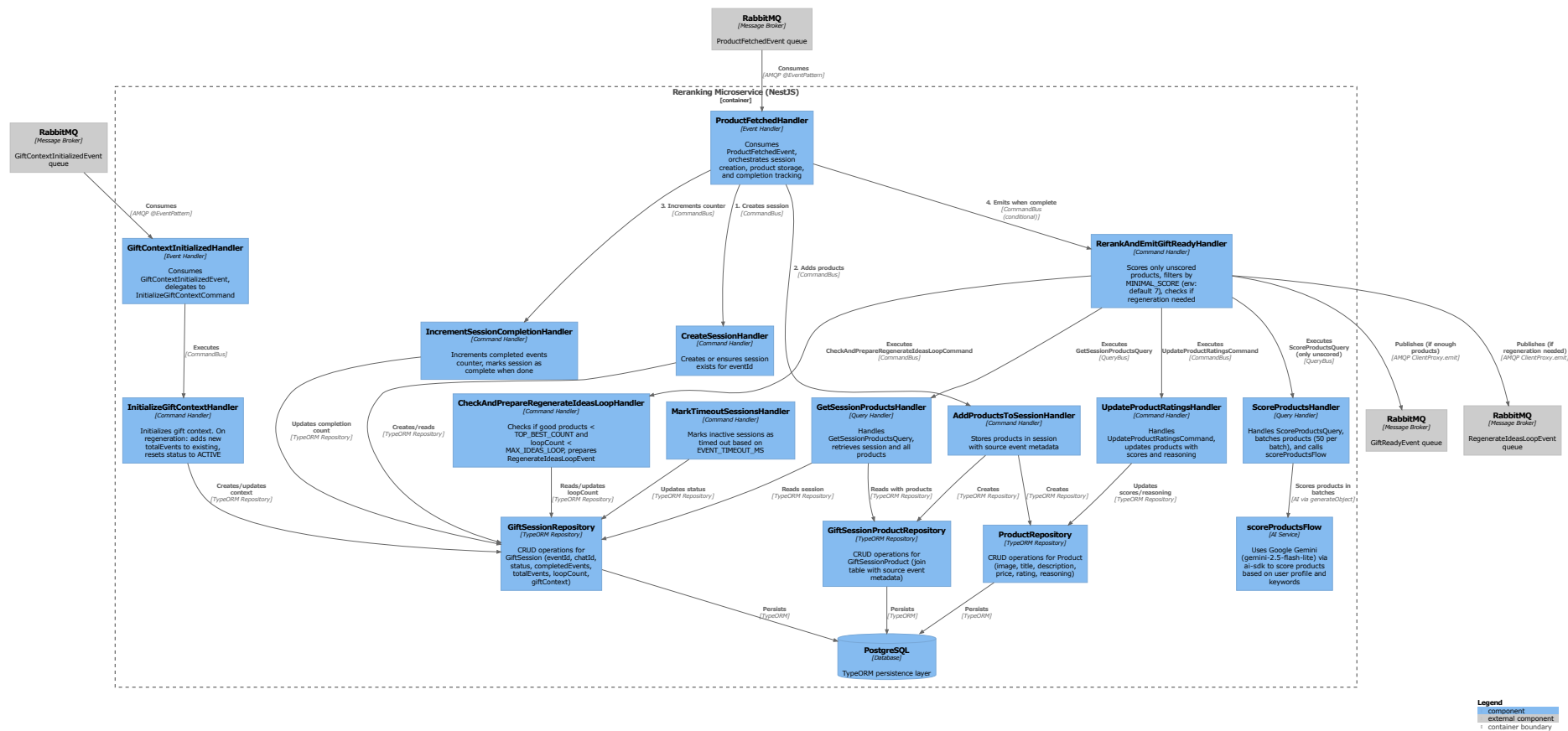
Rysunek 5: Diagram komponentów RestAPI Macroservice (C4 Component).

Chat Microservice. Serwis prowadzący wywiad z użytkownikiem (Rysunek 6) wykorzystuje mechanizm *tool calling*, aby wymusić strukturyzowane odpowiedzi od modelu LLM. Wewnętrznie dzieli się na handler zdarzeń, flow AI oraz repozytorium sesji.



Rysunek 6: Diagram komponentów Chat Microservice (C4 Component).

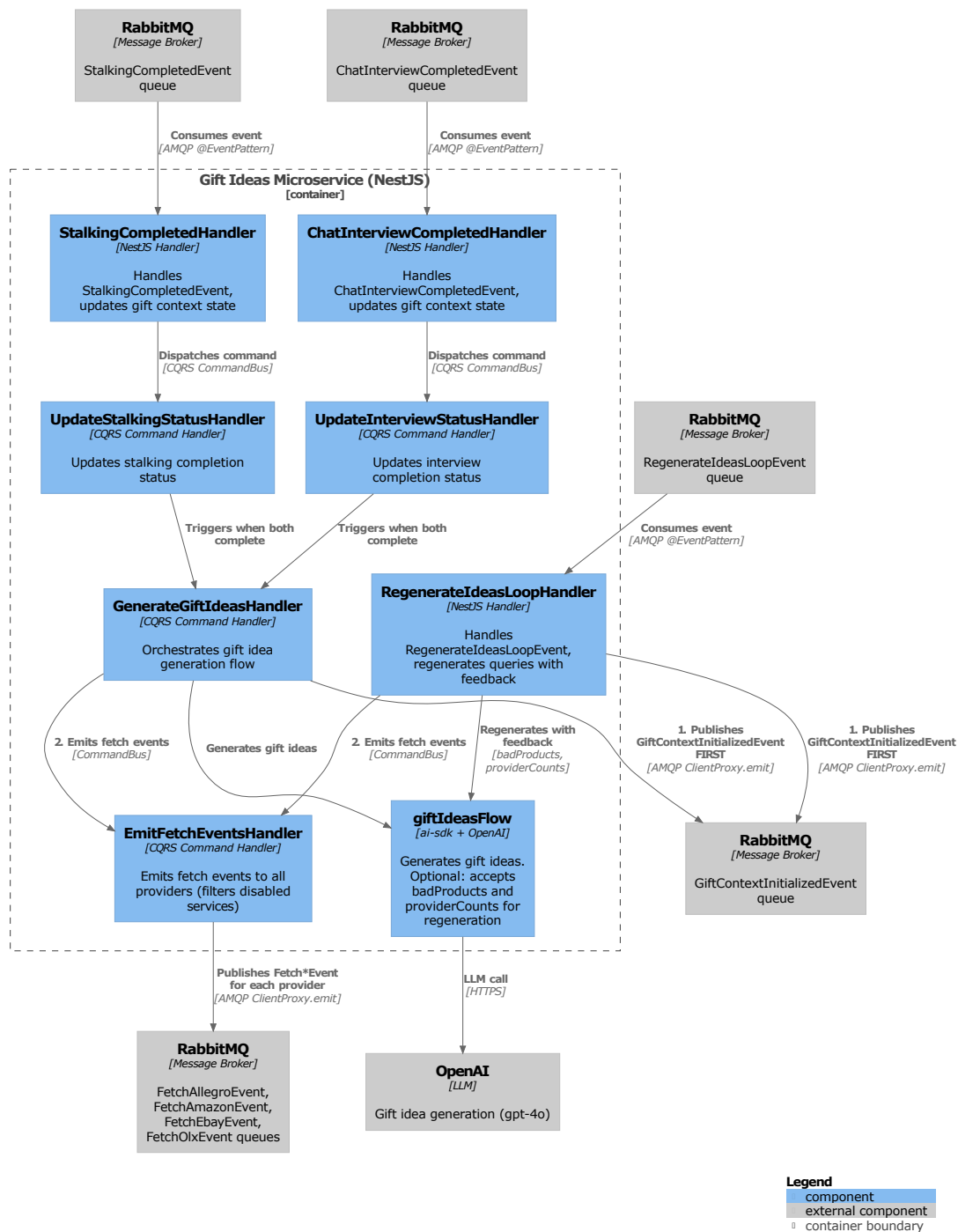
Reranking Microservice. Moduł odpowiedzialny za ocenę i filtrowanie ofert (Rysunek 7) korzysta z modelu Gemini 2.5 do semantycznej analizy każdego produktu. Generuje ocenę punktową oraz tekstowe uzasadnienie, co zapewnia wyjaśnialność decyzji (XAI).



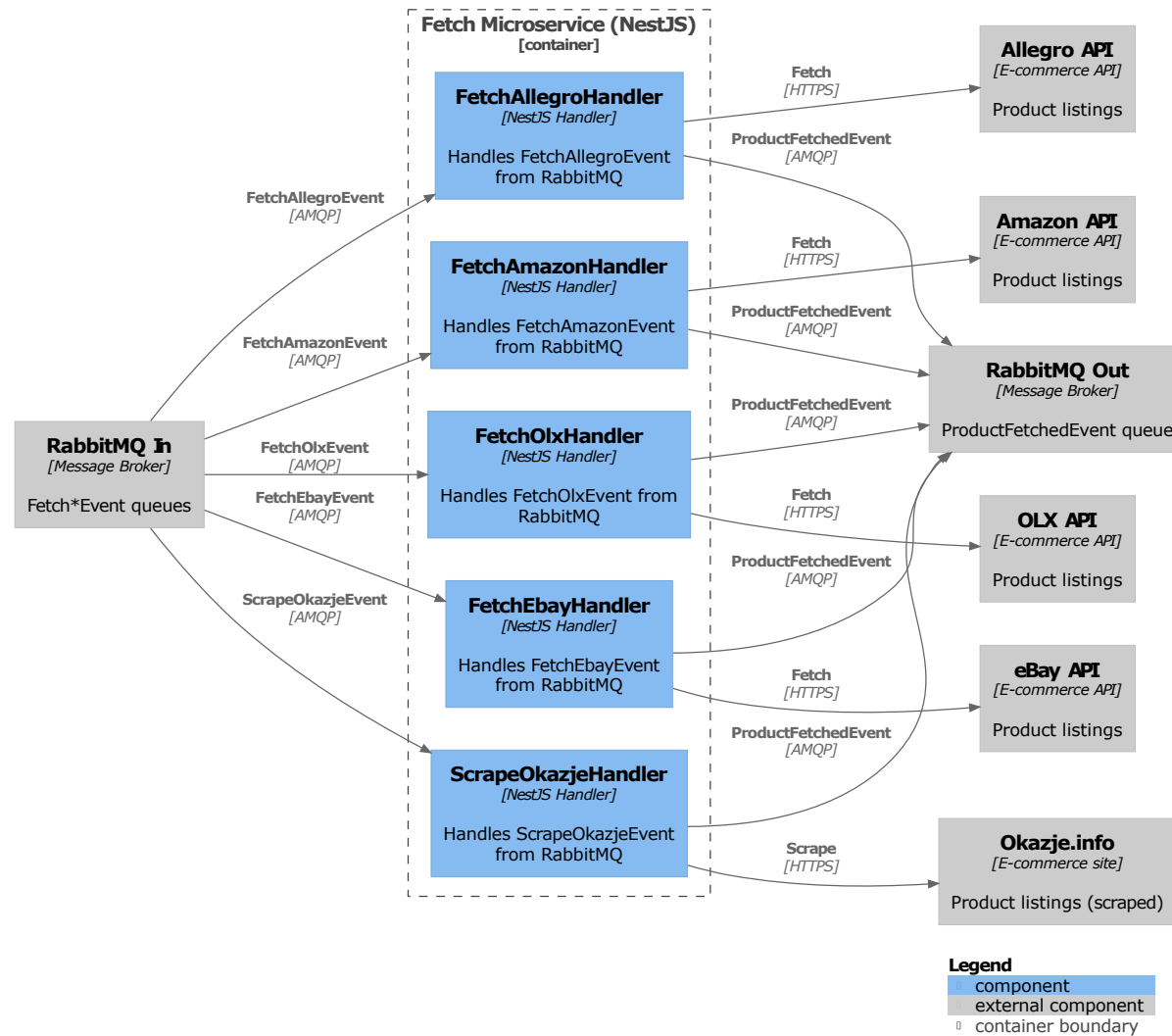
Rysunek 7: Diagram komponentów Reranking Microservice (C4 Component).

Gift Ideas Microservice. Serwis generujący abstrakcyjne pomysły na prezenty na podstawie profilu odbiorcy oraz danych OSINT (Rysunek 8). Odpowiada za tworzenie zapytań dla zintegrowanych platform e-commerce i normalizację wyników przed przekazaniem ich do modułu Fetch.

Fetch Microservice. Moduł odpowiedzialny za agregację ofert z wielu platform (OLX, Allegro, Amazon, eBay, Okazje) na podstawie zapytań wygenerowanych przez serwis Gift Ideas (Rysunek 9). Każde źródło ma dedykowany handler (`FetchOlxHandler`, `FetchAmazonHandler`, `FetchEbayHandler`, `FetchAllegroHandler`) oraz scraper `ScrapeOkazjeHandler` uruchamiany zdarzeniem `ScrapeOkazjeEvent`; wszystkie emitują zunifikowany `ProductFetchedEvent`.



Rysunek 8: Diagram komponentów Gift Ideas Microservice (C4 Component).



Rysunek 9: Diagram komponentów Fetch Microservice (C4 Component).

Stalking Microservice. Serwis realizujący OSINT na profilach społecznościowych użytkownika obdarowywanego (Rysunek 10). Ekstrahuje publiczne fakty i preferencje z trzech źródeł: Instagram, TikTok i X (Twitter), które wzbogacają profil używany przez Gift Ideas i Reranking.

8.5 Zastosowane wzorce projektowe

- **CQRS (Command Query Responsibility Segregation):** Rozdzielenie operacji modyfikujących stan (Command) od zapytań o dane (Query).
- **Event-Driven Architecture:** System reaguje na zdarzenia (np. `StalkingCompletedEvent`, `ProductFetchedEvent`), co pozwala na luźne powiązanie komponentów.
- **Asynchronous Request-Reply:** Obsługa długotrwałych procesów bez blokowania wątku głównego.
- **Strategy:** Wybór odpowiedniego algorytmu pobierania danych w zależności od źródła (np. strategia dla OLX vs strategia dla Amazon).
- **DTO (Data Transfer Object):** Ustrukturyzowane obiekty do przesyłania danych między warstwami.

9 Model danych i baza danych

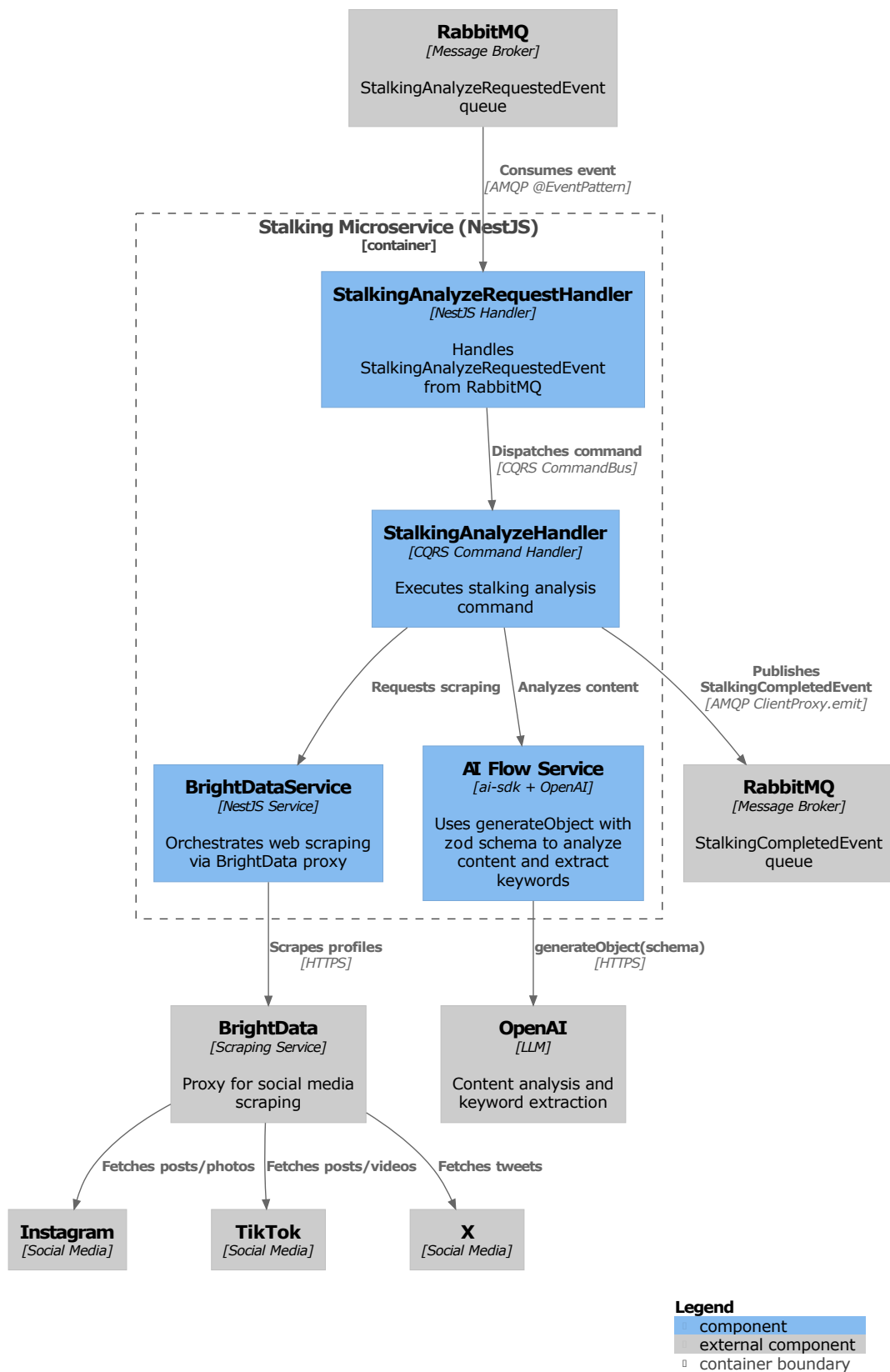
Aby wesprzeć zdefiniowane przypadki użycia i przyjętą architekturę mikroserwisową, zaprojektowano model danych odzwierciedlający główne byty domenowe oraz ich relacje.

9.1 Model konceptualny

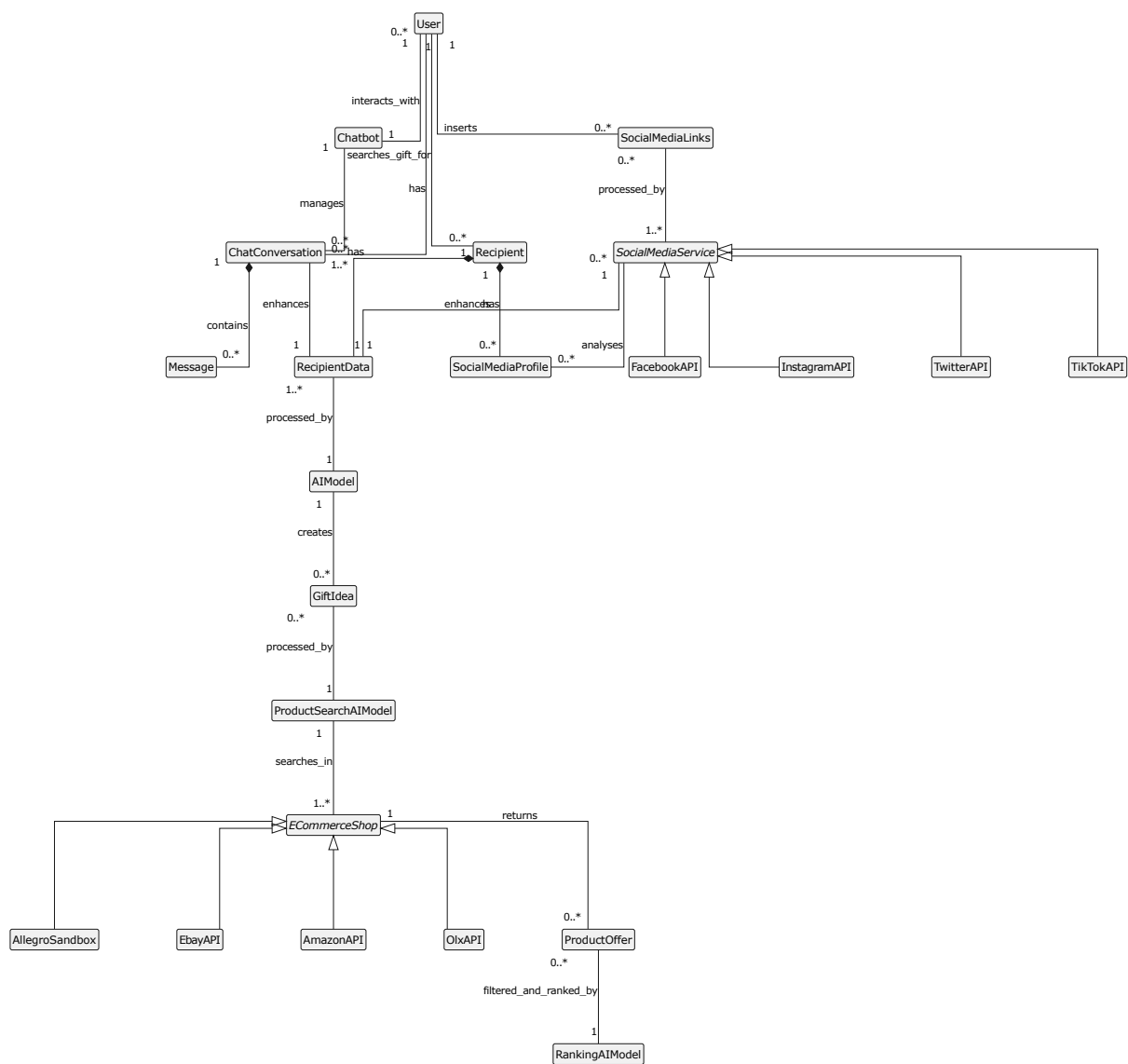
Model danych został zaprojektowany w sposób domenowy, z wykorzystaniem diagramów E/R oraz UML (diagram klas). Główne byty to m.in.:

- **User** – reprezentuje zalogowanego użytkownika systemu (dane z Google OAuth, preferencje),
- **Session** – pojedyncza sesja wyszukiwania prezentów, powiązana z użytkownikiem,
- **ChatMessage** – wiadomości w ramach sesji (pytania i odpowiedzi),
- **GiftIdea** – abstrakcyjny pomysł na prezent wygenerowany przez moduł AI,
- **Product** – konkretna oferta produktu pobrana z serwisu e-commerce,
- **Feedback** – ocena użytkownika dotycząca trafności rekomendacji,
- **Favorite** – relacja użytkownika do wybranych produktów (lista ulubionych).

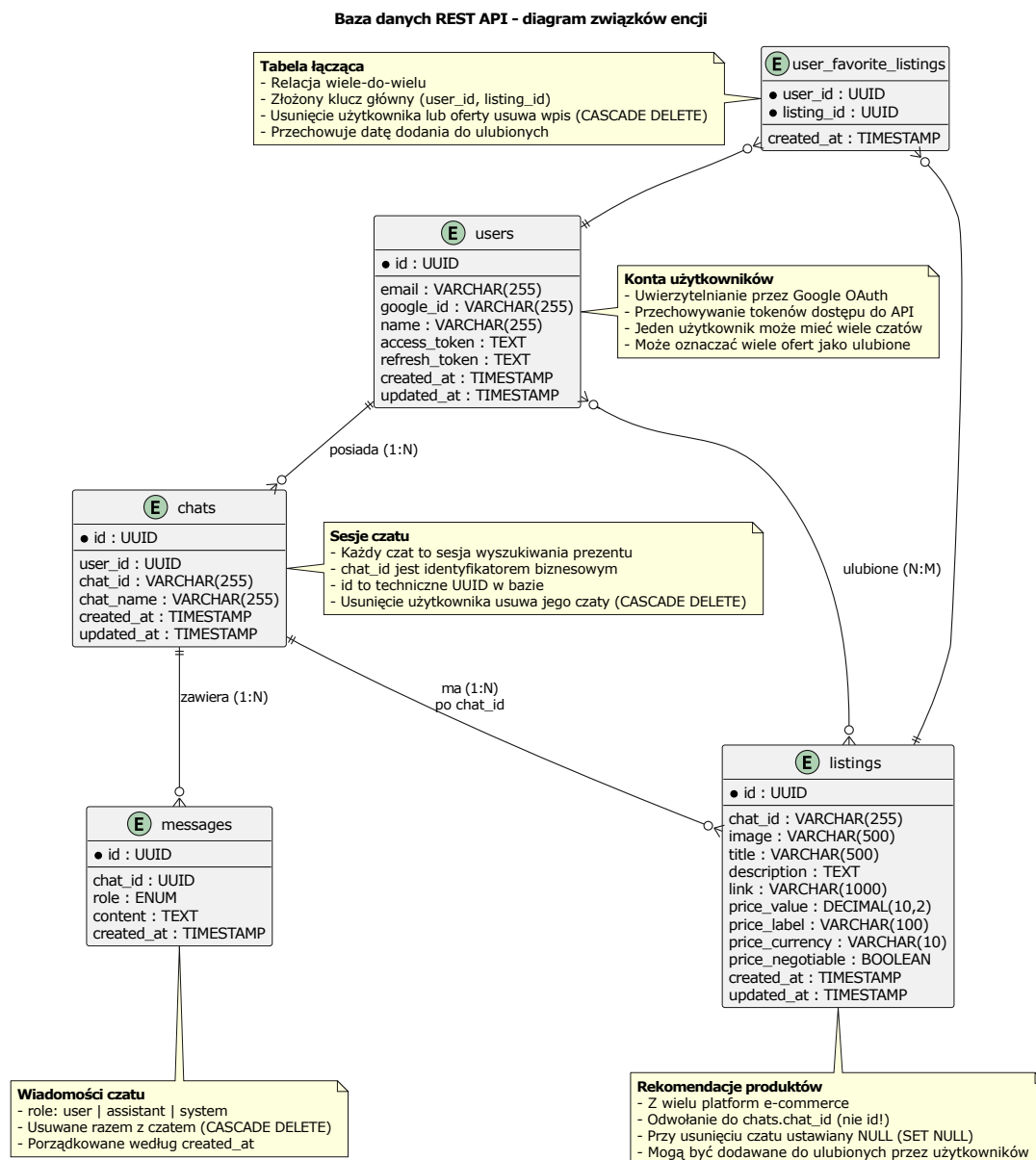
Relacje między tymi bytami zostały odwzorowane na diagramie konceptualnym (User posiada wiele Session; Session posiada wiele ChatMessage i GiftIdea; GiftIdea powiązana jest z wieloma Product; User może dodawać Product do Favorites oraz pozostawiać Feedback dla Session). Na Rysunku 11 przedstawiono diagram klas UML modelu domenowego, natomiast diagram E/R pokazano na Rysunku 12. W dalszej części rozdziału umieszczono również fizyczny diagram bazy RestAPI (Rysunek 13) oraz schematy baz mikroserwisów Chat, Gift Ideas i Reranking (Rysunki 14, 15, 16).



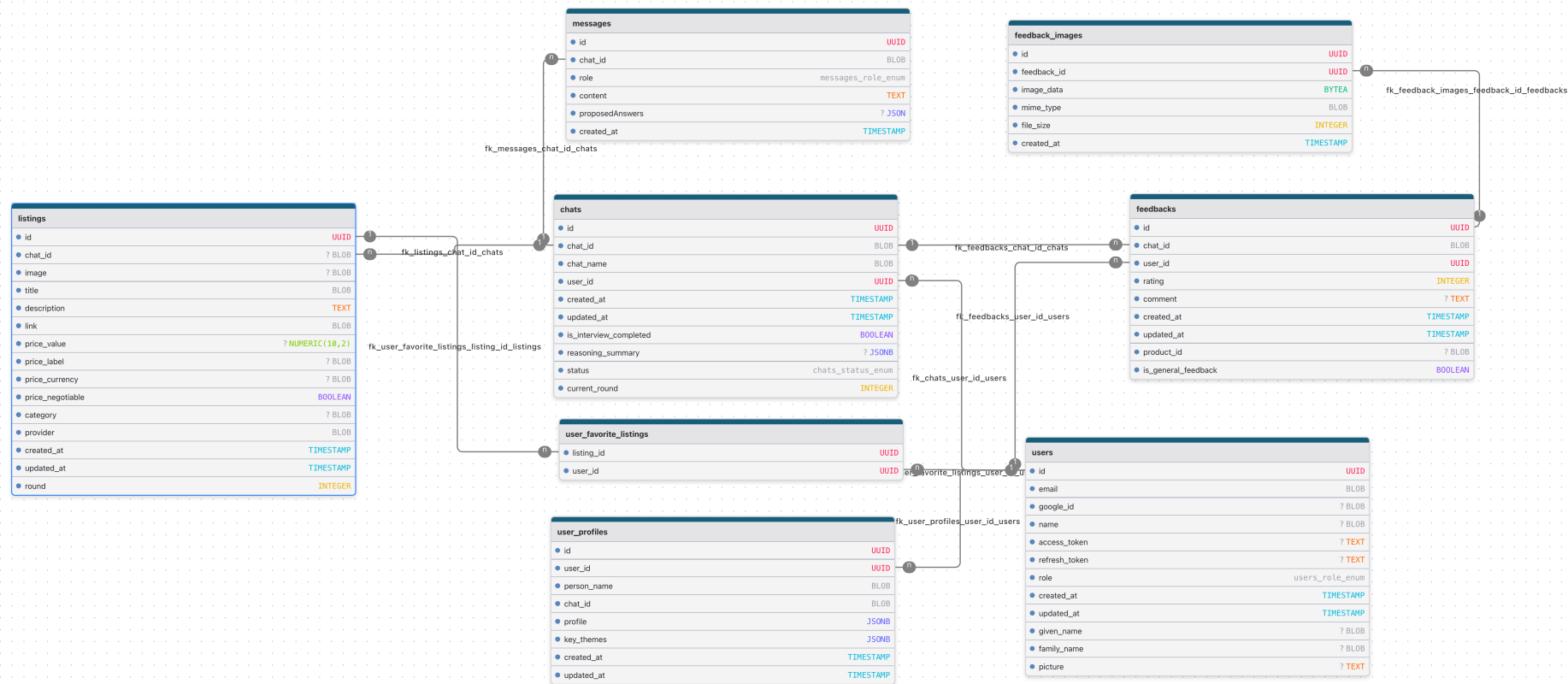
Rysunek 10: Diagram komponentów Stalking Microservice (C4 Component).



Rysunek 11: Diagram klas UML modelu domenowego systemu.



Rysunek 12: Diagram E/R modelu danych głównej bazy (RestAPI).



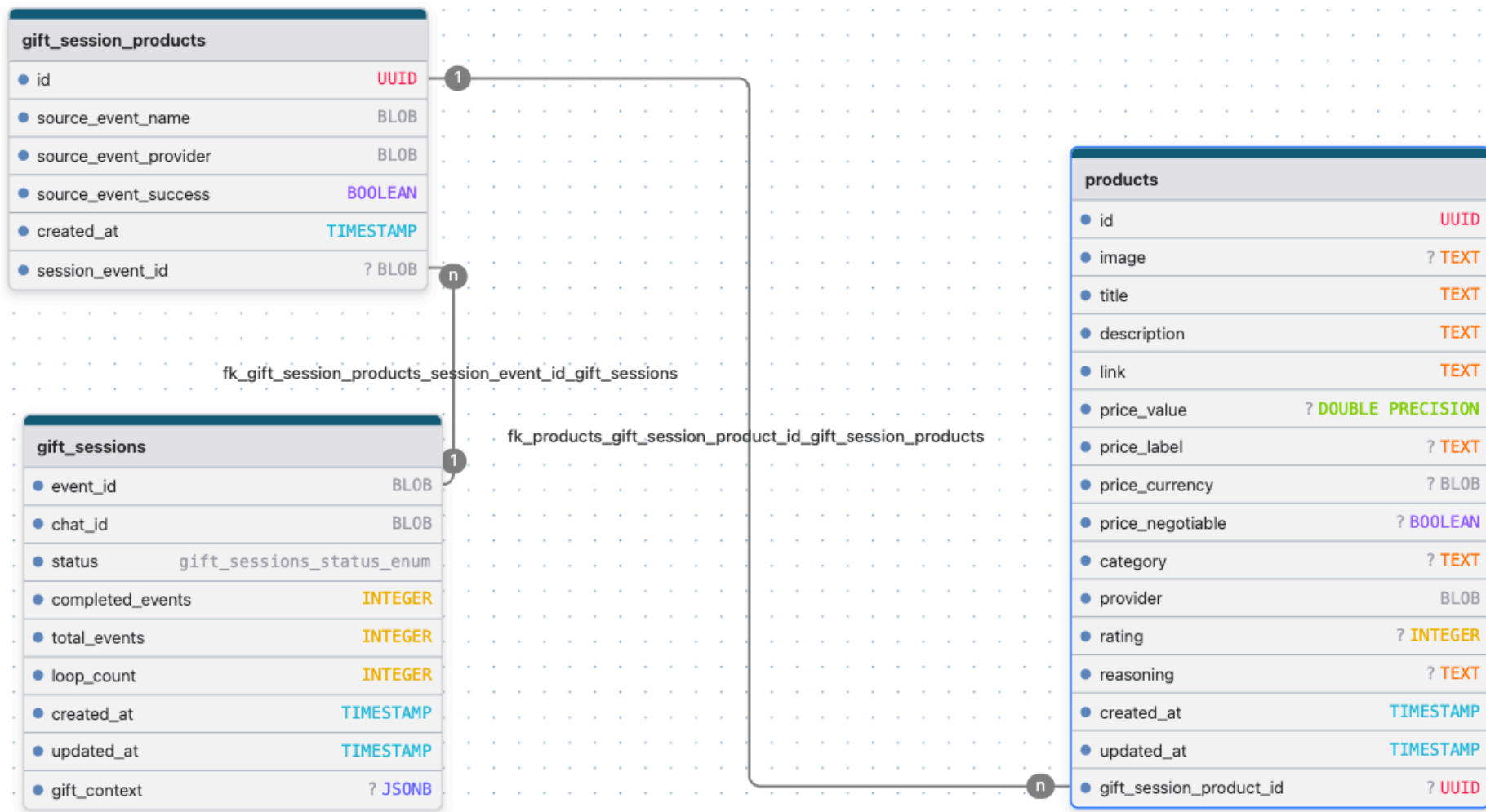
Rysunek 13: Diagram fizyczny bazy danych mikroserwisu RestAPI.

chat_sessions	
● chat_id	BLOB
● occasion	? BLOB
● phase	BLOB
● pending_profile_data	? JSONB
● save_profile_choice	? BOOLEAN
● created_at	TIMESTAMP
● updated_at	TIMESTAMP
● selected_listing_ids	? JSONB
● selected_listings_context	? JSONB
● refinement_count	INTEGER

Rysunek 14: Schemat bazy danych mikroserwisu *Chat*.

chat_sessions	
● chat_id	BLOB
● stalking_status	chat_sessions_stalking_status_enum
● interview_status	chat_sessions_interview_status_enum
● stalking_keywords	? JSONB
● interview_profile	? JSONB
● interview_keywords	? JSONB
● gift_generation_triggered	BOOLEAN
● save_profile	? BOOLEAN
● profile_name	? BLOB
● created_at	TIMESTAMP
● updated_at	TIMESTAMP
● min_price	? NUMERIC(10,2)
● max_price	? NUMERIC(10,2)

Rysunek 15: Schemat bazy danych mikroserwisu *Gift Ideas*.

Rysunek 16: Schemat bazy danych mikroserwisu *Reranking*.

9.2 Implementacja w DBMS

Implementację modelu danych wykonano w PostgreSQL z wykorzystaniem TypeORM. Dla każdej encji domenowej przygotowano odpowiadającą jej tabelę, z kluczami głównymi typu UUID oraz indeksami przyspieszającymi wyszukiwanie po polach takich jak `userId`, `sessionId` czy `provider` (OLX, Allegro, eBay, Amazon).

Początkowo używano trybu `synchronize: true`, jednak na etapie wdrożenia produkcyjnego przełączono się na migracje bazodanowe, aby zapewnić kontrolę nad zmianami schematu. Migracje generowano narzędziem CLI TypeORM i uruchamiano w pipeline CI/CD oraz na serwerze produkcyjnym.

W dokumentacji bazy danych wyróżniono widok całościowy (diagram wszystkich tabel) oraz zbliżenia na ważniejsze fragmenty, takie jak model sesji i historii wyszukiwań czy model przechowywania feedbacku użytkowników.

Zdefiniowany model danych stanowi statyczny obraz systemu. Aby w pełni zrozumieć jego działanie, w kolejnym rozdziale przedstawiono modelowanie behawioralne, opisujące dynamikę interakcji i przepływów.

10 Modelowanie behawioralne

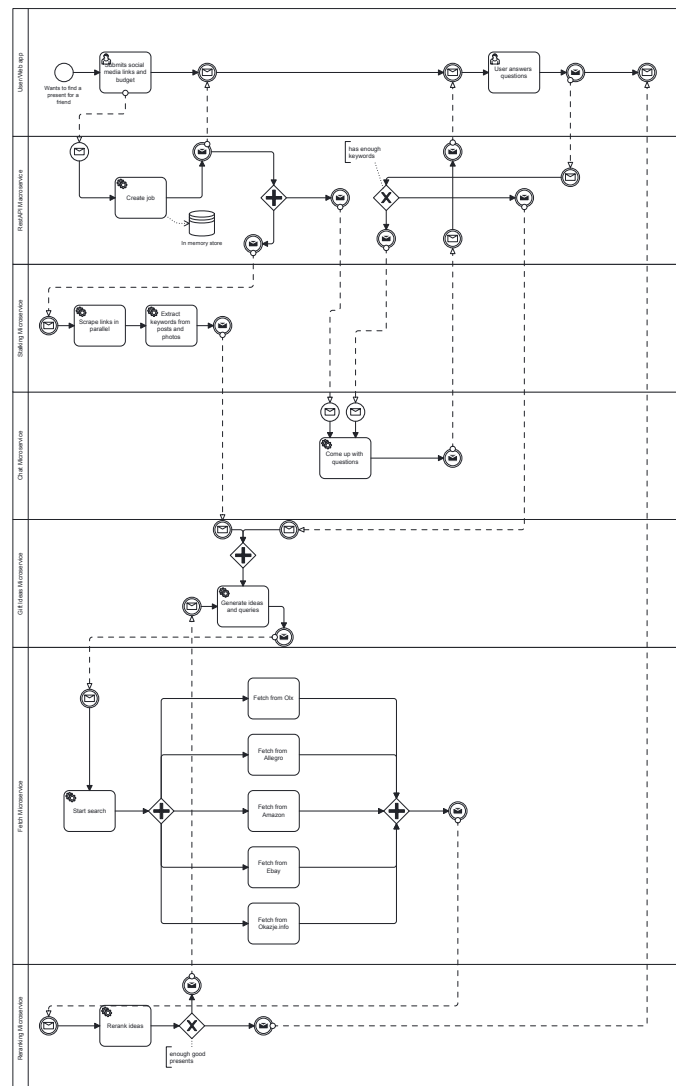
Po ustaleniu struktury systemu istotne było opisanie jego zachowania w czasie w postaci sekwencji interakcji, stanów oraz procesów biznesowych, które prowadzą użytkownika od podania linku aż do listy konkretnych ofert prezentów.

10.1 Diagram sekwencji

Dla głównego scenariusza użycia (wyszukiwanie prezentu) przygotowano diagram sekwencji UML (Rysunek 17) przedstawiający przepływ wiadomości pomiędzy: Użytkownikiem, Frontendem, RestAPI Macroservice, Chat Microservice, Stalking Microservice, Gift Ideas Microservice, Fetch Microservice oraz Reranking Microservice. Diagram pokazuje m.in. moment, w którym Chat Microservice decyduje o rozpoczęciu wyszukiwania oraz sposób, w jaki wyniki są strumieniowane do użytkownika przez SSE.

10.2 BPMN

Proces biznesowy od momentu wejścia użytkownika na stronę (landing page) do zakończenia sesji wyszukiwania został odwzorowany w notacji BPMN (Rysunek 18). Diagram przedstawia m.in. decyzje użytkownika (kontynuować wywiad vs. przejść od razu do wyników), punkty integracji z zewnętrznymi API oraz miejsca potencjalnych błędów (np. brak odpowiedzi od dostawcy), wraz z obsługą wyjątków.



Rysunek 18: Diagram BPMN procesu wyszukiwania prezentu w systemie.

Sformalizowane procesy biznesowe i diagramy aktywności stały się bezpośrednim wsadem do etapu projektowania interfejsu użytkownika.

11 Prototypowanie interfejsu

Na podstawie powyższych modeli zaprojektowano interfejs użytkownika, który prowadzi odbiorcę przez najważniejsze przepływy w możliwie naturalny i zrozumiały sposób.

Interfejs użytkownika powstawał iteracyjnie, w ścisłym powiązaniu z modelem wymagań i architektury. W narzędziu Figma przygotowano prototypy widoków, które następnie przekładano na komponenty React (Shaden UI + Tailwind), zachowując spójność z przepływami zdefiniowanymi na diagramach C4, UML i BPMN.

Poniżej przedstawiono pełne flow użytkownika – od wejścia na stronę, przez konfigurację wyszukiwania i wywiad z chatbotem, aż do otrzymania spersonalizowanych rekomendacji oraz zarządzania zapisanymi produktami.

11.1 Krok 1: Strona główna (Landing Page)

Użytkownik trafia na stronę główną (Rysunek 19a), która w prosty sposób przedstawia wartość aplikacji: „Znajdź idealny prezent w kilka minut”. Strona składa się z kilku sekcji:

- **Hero** – główny baner z logo i przyciskiem CTA „Rozpocznij”,
- **Jak to działa** – 4 karty opisujące proces (linki społecznościowe → rozmowa z AI → wyszukiwanie → filtrowanie),
- **Zespół** – informacje o twórcach aplikacji,
- **FAQ** – odpowiedzi na najczęściej zadawane pytania.

Na tym etapie użytkownik może zalogować się przez Google OAuth, co umożliwia zapisywanie historii sesji i ulubionych produktów.

11.2 Krok 2: Formularz wyszukiwania

Po kliknięciu przycisku „Rozpocznij” użytkownik przechodzi do formularza konfiguracji wyszukiwania (Rysunek 19b). Formularz składa się z trzech sekcji:

a) Linki do mediów społecznościowych (opcjonalne) – użytkownik może podać adresy profili Instagram, X (Twitter) lub TikTok obdarowywanej osoby. System wykorzysta publiczne dane do lepszego zrozumienia zainteresowań odbiorcy.

b) Wybór okazji – siatka 4 przycisków pozwala określić kontekst prezentu: Urodziny, Rocznica, Święta, Tak po prostu.

c) Budżet – użytkownik wybiera przedział cenowy (do 50 zł, 50–100 zł, 100–200 zł) lub podaje własny zakres.

AI Present Finder

Znajdź idealne pomysły na prezenty dla bliskich dzięki spersonalizowanym rekomendacjom napędzanym przez AI.

Rozpocznij



Jak to działa -

(a) Strona główna (landing page).

Znajdź idealny prezent

Opowiedz nam o tej osobie

Wpisz adres URL jej profilu w mediach społecznościowych. Obsługujemy Instagram, X i TikTok.

 instagram.com/username

 x.com/username

 tiktok.com/@username

Detale

Jaka jest okazja?



Urodziny



Rocznica

Znajdź pomysły na prezenty



Szukaj



Zapisane



Historia



Profil

Rysunek 19: Początek flow użytkownika: strona główna i formularz konfiguracji wyszukiwania.

Jeśli użytkownik wcześniej szukał prezentu dla tej samej osoby, system zaproponuje wczytanie zapisanego profilu, co przyspiesza proces konfiguracji.

11.3 Krok 3: Wywiad z chatbotem

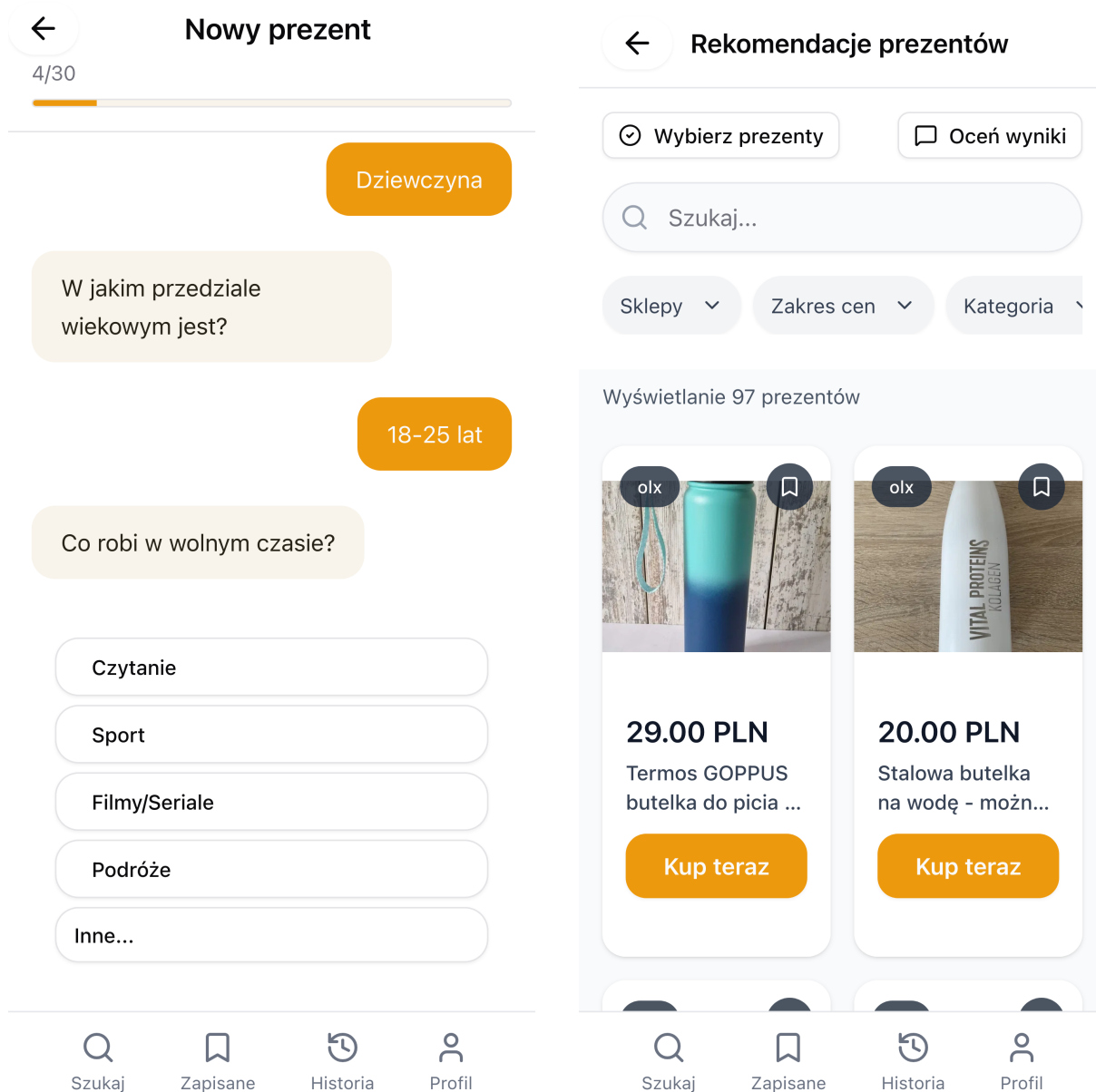
Po wysłaniu formularza użytkownik przechodzi do interfejsu chatu (Rysunek 20a). Agent AI prowadzi krótki wywiad, dopytując o szczegóły dotyczące relacji z obdarowywaną osobą, zainteresowań i hobby odbiorcy oraz preferencji stylistycznych.

Chatbot oferuje sugerowane odpowiedzi (przyciski), ale użytkownik może też wpisać własną odpowiedź. Pasek postępu w nagłówku informuje o etapie wywiadu. Cała rozmowa trwa średnio 2–3 minuty i składa się z ok. 8–12 pytań.

11.4 Krok 4: Lista rekomendacji

Po zakończeniu wywiadu system analizuje zebrane dane, generuje abstrakcyjne pomysły na prezenty, a następnie wyszukuje konkretne oferty w czterech serwisach e-commerce (OLX, Allegro, Amazon, eBay).

Wyniki są prezentowane w formie przejrzystej siatki kart (Rysunek 20b), gdzie każda oferta zawiera zdjęcie produktu, badge z nazwą platformy, tytuł, cenę oraz przycisk „Kup teraz”. Górna belka oferuje rozbudowane opcje filtrowania oraz tryby doprecyzowania i oceniania.



(a) Interfejs chatu z AI.

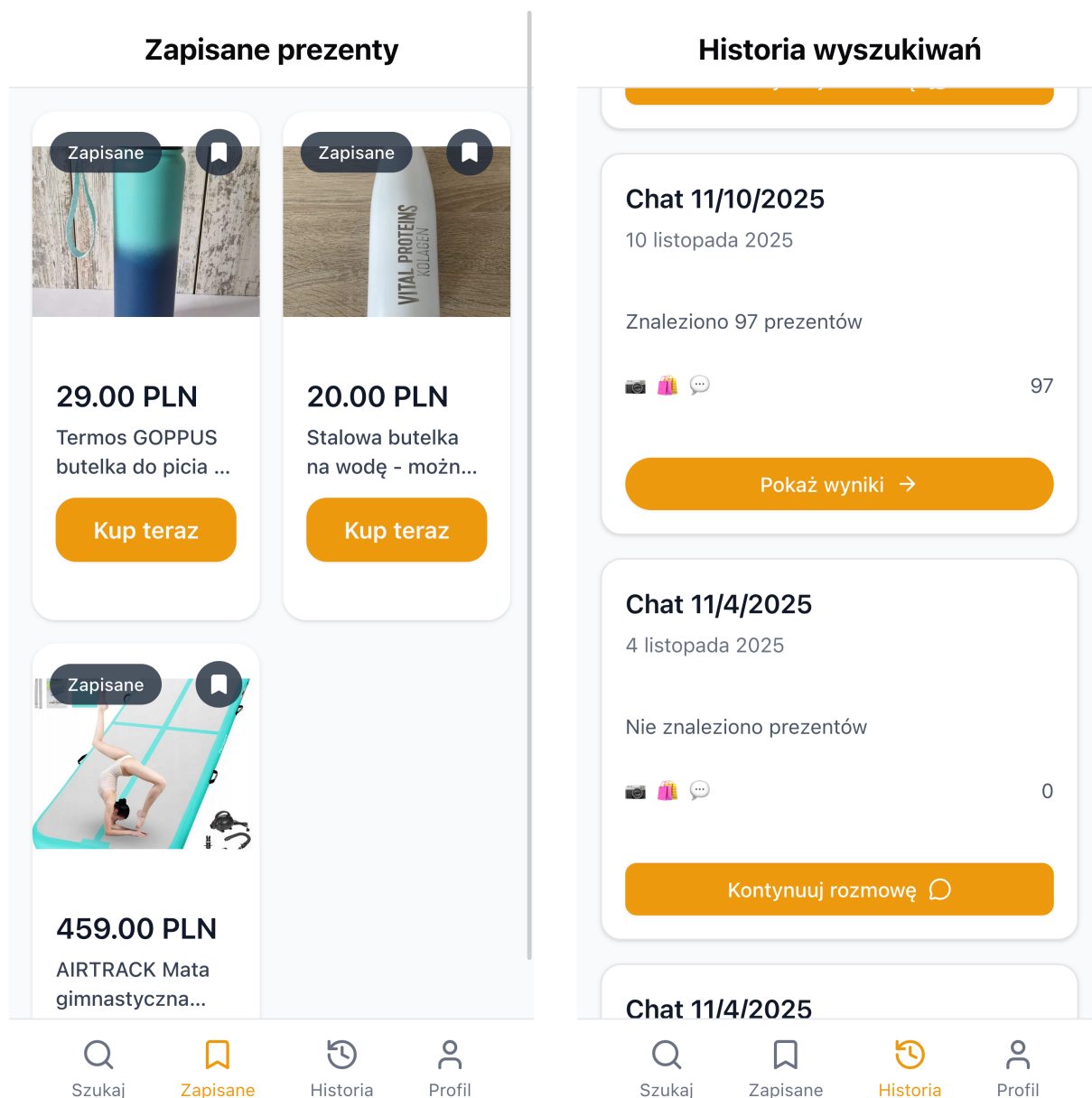
(b) Lista rekomendacji prezentów.

Rysunek 20: Główny flow: wywiad z chatbotem i wynikowa lista rekomendacji.

11.5 Krok 5: Nawigacja i zarządzanie

Dolna belka nawigacyjna (widoczna na wszystkich ekranach po zalogowaniu) zapewnia szybki dostęp do czterech głównych sekcji aplikacji:

- a) **Szukaj** – powrót do formularza nowego wyszukiwania.
- b) **Zapisane** (Rysunek 21a) – lista ulubionych produktów oznaczonych ikoną zakładki podczas przeglądania rekomendacji.
- c) **Historia** (Rysunek 21b) – archiwum poprzednich sesji wyszukiwania z możliwością powrotu do wyników lub kontynuacji niedokończonej rozmowy.



(a) Zapisane (ulubione) produkty.

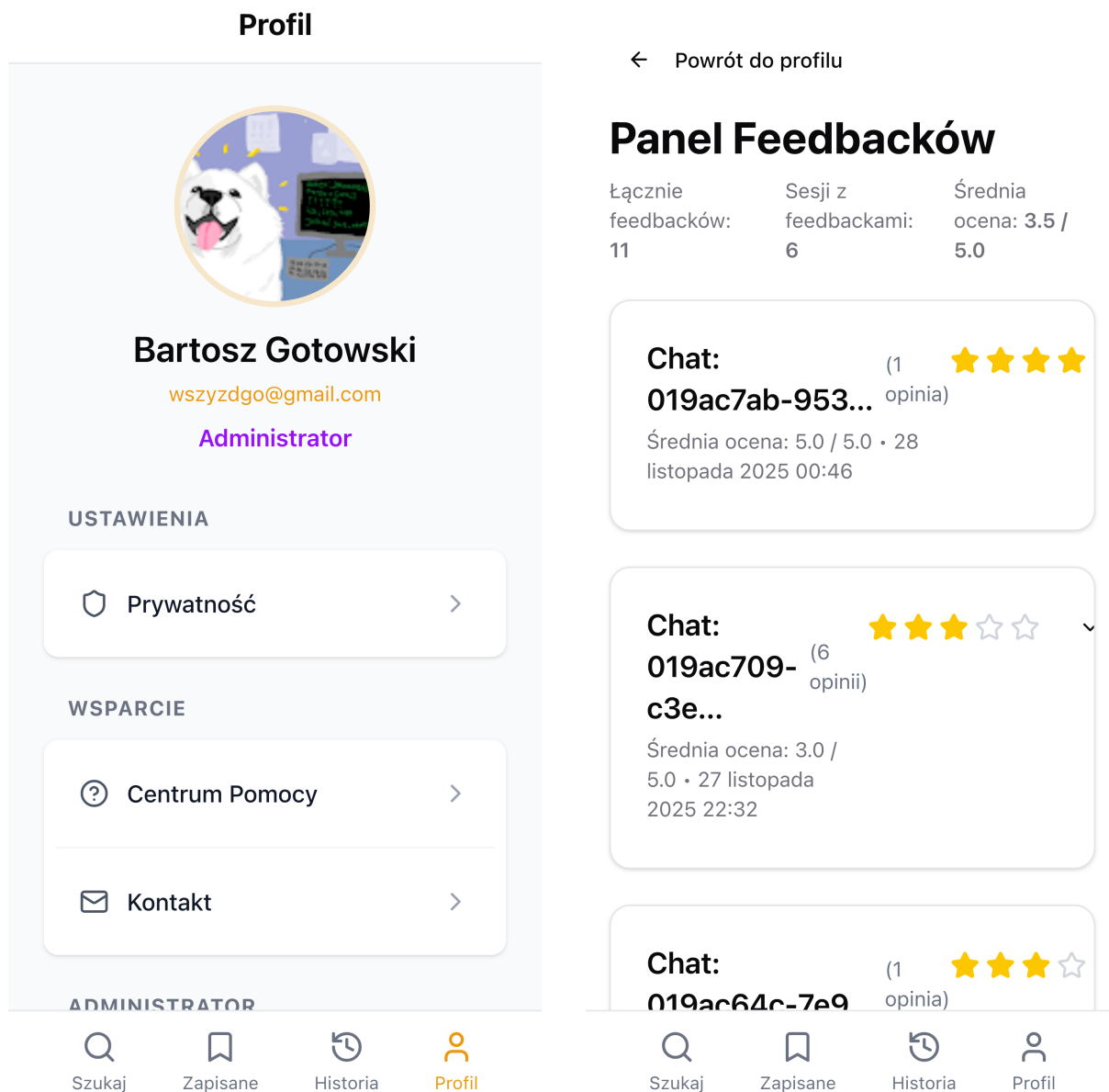
(b) Historia sesji wyszukiwania.

Rysunek 21: Zarządzanie danymi: zapisane produkty i historia sesji.

d) **Profil** (Rysunek 22a) – zarządzanie kontem użytkownika zawierające zdjęcie, dane, ustawienia konta oraz przycisk wylogowania. Administratorzy mają dodatkowo dostęp do panelu feedbacków.

11.6 Panel administracyjny

Użytkownicy z rolą administratora mają dostęp do panelu feedbacków (Rysunek 22b), gdzie mogą przeglądać wszystkie opinie użytkowników pogrupowane według sesji. Panel wyświetla statystyki: łączną liczbę feedbacków, liczbę sesji z opiniami oraz średnią ocenę.



(a) Panel profilu użytkownika.

(b) Panel administracyjny – feedbacki.

Rysunek 22: Profil użytkownika i panel administracyjny.

Zatwierdzone projekty interfejsu oraz architektura systemu pozwoliły na przejście do fazy implementacji, w której zrealizowano najważniejsze mechanizmy, w tym algorytmy AI.

12 Implementacja

12.1 Algorytm Rerankingu z Weryfikacją (XAI)

Najważniejszym elementem systemu jest moduł rerankingu, który filtruje oferty niedopasowane do profilu odbiorcy. Tradycyjne wyszukiwanie oparte na prostych frazach tekstowych często zwraca wyniki nieadekwatne (np. akcesoria do konsoli zamiast konsoli). Zaimplementowany algorytm wykorzystuje model LLM (Gemini 2.5) do oceny semantycznej każdego produktu. Dla każdej oferty model generuje:

- Ocenę punktową (0-10) zgodności z profilem odbiorcy.
- Tekstowe uzasadnienie decyzji (np. "Odrzucono: Produkt to część zamienna, a nie gotowy prezent").

Pozwala to na odfiltrowanie ok. 40% szumu informacyjnego.

Poniżej przedstawiono fragment systemu promptów definiującego kryteria oceny produktów:

Jak pokazano w Listingu 1, kryteria oceny produktów są zapisane w postaci czytelnego dla człowieka systemu promptów, który wymusza wyjaśnialne (XAI) decyzje modelu.

Listing 1: Fragment systemu promptów dla rerankingu produktów

```
1 <criteria>
2   <relevance priority="highest">
3     Czy produkt odpowiada zainteresowaniom i hobby odbiorcy?
4   </relevance>
5   <lifestyle_fit priority="high">
6     Czy produkt pasuje do stylu życia i codziennej rutyny?
7   </lifestyle_fit>
8   <preferences priority="high">
9     Czy produkt jest zgodny z preferencjami estetycznymi?
10  </preferences>
11  <occasion priority="medium">
12    Czy produkt jest odpowiedni dla danej okazji?
13  </occasion>
14 </criteria>
15
16 <scoring>
17   1-3: Slabe dopasowanie - produkt nie pasuje do profilu
18   4-6: Srednie dopasowanie - moze pasowac, ale nie jest idealny
19   7-8: Dobre dopasowanie - produkt dobrze pasuje do profilu
20   9-10: Doskonale dopasowanie - idealnie pasuje do profilu i okazji
21
22   Wyniki ponizej 5 nie beda widoczne uzytkownikowi.
23 </scoring>
```

12.2 Orkiestracja zdarzeń w architekturze CQRS

Centralnym elementem systemu jest handler `StartProcessingCommand`, który orkiestruje rozpoczęcie całego procesu wyszukiwania prezentu. Po otrzymaniu żądania od użytkownika tworzy sesję chatu w bazie danych, a następnie równolegle publikuje dwa zdarzenia

do kolejki RabbitMQ: jedno uruchamia analizę OSINT profili społecznościowych, drugie – wywiad z chatbotem.

Listing 2 przedstawia uproszczoną implementację tego handlera, podkreślając zasadę: najpierw utrwalenie sesji w bazie, dopiero później emisja zdarzeń do innych mikroserwisów.

Listing 2: Orkiestracja zdarzeń przy rozpoczęciu wyszukiwania prezentu

```
1 @CommandHandler(StartProcessingCommand)
2 export class StartProcessingCommandHandler {
3   constructor(
4     @Inject("STALKING_ANALYZE_REQUESTED_EVENT")
5     private readonly stalkingEventBus: ClientProxy,
6     @Inject("CHAT_START_INTERVIEW_EVENT")
7     private readonly chatEventBus: ClientProxy,
8     private readonly chatRepository: IChatRepository,
9   ) {}
10
11   async execute(command: StartProcessingCommand) {
12     // 1. Najpierw tworzymy sesję w bazie danych
13     const chat = await this.chatRepository.create({
14       chatId: analyzeRequestedDto.chatId,
15       chatName: `Chat ${new Date().toLocaleDateString()}`,
16       userId,
17     });
18
19     // 2. Dopiero po utrwaleniu emitujemy zdarzenia równolegle
20     this.stalkingEventBus.emit(
21       StalkingAnalyzeRequestedEvent.name, stalkingEvent
22     );
23     this.chatEventBus.emit(
24       ChatStartInterviewEvent.name, interviewEvent
25     );
26   }
27 }
```

12.3 Strukturyzacja rozmowy przez Tool Calling

Moduł chatu wykorzystuje mechanizm *tool calling* do wymuszenia strukturyzowanej odpowiedzi od modelu LLM. Zamiast swobodnego tekstu, AI musi wywołać jedno z trzech narzędzi: zadać pytanie z sugerowanymi odpowiedziami, zakończyć wywiad z wygenerowanym profilem, lub oznaczyć nieodpowiednie zapytanie.

Listing 3 pokazuje definicję tych narzędzi wraz z odpowiadającymi im schematami Zod, co zapewnia typowaną i przewidywalną współpracę pomiędzy LLM a resztą systemu.

Listing 3: Definicja narzędzi AI dla strukturyzacji wywiadu

```
1 export const tools = {
2   ask_a_question_with_answer_suggestions: tool({
3     description: "Zadaj pytanie z 4 sugerowanymi odpowiedziami",
4     inputSchema: z.strictObject({
5       question: z.string().min(1),
```

```

6     potentialAnswers: potencialAnswersSchema,
7   },
8 },
9
10  end_conversation: tool({
11    description: "Zakończ rozmowę z wygenerowanym profilem",
12    inputSchema: z.object({
13      output: endConversationOutputSchema, // profil odbiorcy
14    }),
15  }),
16
17  flag_inappropriate_request: tool({
18    description: "Oznacz nieetyczne lub nielegalne zapytanie",
19    inputSchema: z.object({
20      reason: z.string(),
21    }),
22  }),
23 } as const;

```

Dzięki temu podejściu każda odpowiedź AI jest typowana i walidowana schematem Zod, co eliminuje problemy z parsowaniem i zapewnia spójność przepływu danych między mikroservisami.

12.4 Inżynieria promptów (Prompt Engineering)

Jednym z najważniejszych aspektów implementacji systemu AI Present Finder była iterycyjna optymalizacja promptów – instrukcji przekazywanych modelom językowym. W ciągu 80 dni rozwoju projektu (wrzesień–listopad 2025) przeprowadzono ponad 25 iteracji promptów, co przełożyło się na znaczącą poprawę jakości generowanych rekomendacji. Podsumowanie zmian znajduje się w Tabeli 3.

Ewolucja promptów. System wykorzystuje 5 specjalizowanych promptów, z których każdy przeszedł własną ścieżkę optymalizacji:

Prompt	Iteracje	Kluczowe zmiany
Chat (wywiad)	12	Format XML, min. 30 pytań, 3. osoba, podejście produktowe
Refinement	2	Doprecyzowanie po wyborze produktów (3–5 pytań)
Stalking (OSINT)	3	Ekstrakcja faktów z profili społecznościowych
Gift Ideas	5	Generowanie zapytań dla 5 platform, max 5 wyrazów
Reranking	4	Scoring 1–10, filtrowanie ≥ 5 , batching

Tabela 3: Przegląd promptów i liczba iteracji w projekcie

Kluczowe problemy i rozwiązania. Podczas rozwoju systemu zidentyfikowano i rozwiązano szereg problemów związanych z zachowaniem modeli LLM:

- **Za krótkie rozmowy:** Początkowy prompt generował tylko 10 pytań, co dawało niewystarczające dane. Rozwiązanie: dodanie licznika pytań i wymuszenie minimum 30 pytań przed zakończeniem wywiadu.
- **Pytania w 2. osobie:** Model pytał „Czy lubisz gotować?” zamiast „Czy ON/ONA lubi gotować?”. Rozwiązanie: explicite reguła w prompcie wymuszająca 3. osobę.
- **Zbyt abstrakcyjne pytania:** Pytania typu „Jaki styl preferuje?” nie prowadziły do konkretnych kategorii produktów. Rozwiązanie: podejście produktowe – „Czy ma dobre słuchawki?” zamiast „Jaki rodzaj muzyki lubi?”.
- **Brak różnorodności:** Model zawsze zaczynał od pracy zawodowej. Rozwiązanie: lista 16 obszarów eksploracji z wymogiem pokrycia min. 5 różnych.
- **Drażnienie po „nie wiem”:** Model kontynuował ten sam wątek mimo braku wiedzy użytkownika. Rozwiązanie: reguła natychmiastowej zmiany obszaru po odpowiedzi „nie wiem”.
- **Brakujące tool calls:** Model czasami odpowiadał tekstem zamiast wywołać narzędzie. Rozwiązanie: mechanizm auto-retry z przypomnieniem w kolejnej wiadomości.

Struktura prompta wywiadu. Największy prompt (Chat) liczy ponad 700 linii i wykorzystuje format XML dla lepszej czytelności przez model. Listing 4 przedstawia uproszczoną strukturę tego prompta.

Listing 4: Struktura prompta wywiadu z użytkownikiem (uproszczony)

```

1 <system>
2   <role>Jestes Doradca Prezentowym - prowadzisz rozmowe
3     (15-30 pytan) aby poznać obdarowywanego.</role>
4
5   <context>
6     <occasion>${occasion}</occasion>
7     <conversation_progress>
8       <current_question_number>${questionCount}</current_question_number>
9       <status>MUSISZ ZADAC PRZYNAJMNIEJ ${30 - questionCount}
10        PYTAN WIECEJ!!!</status>
11     </conversation_progress>
12   </context>
13
14   <critical_rules>
15     <rule id="1">JEDNO pytanie na raz, PROSTE, konkretne</rule>
16     <rule id="2">TRZECIA osoba (on/ona) - NIGDY druga osoba</rule>
17     <rule id="3">Pytaj PRODUKTOWO, nie abstrakcyjnie</rule>
18     <rule id="5">Eksploruj MINIMUM 5 ROZNYCH obszarow zycia</rule>
19     <rule id="6">"Nie wiem" = NATYCHMIAST zmien watek</rule>
20   </critical_rules>
21
22   <exploration_leads>
23     PRACA, DOM, HOBBY, KULINARIA, TECHNOLOGIA, ZDROWIE,
24     PODROZE, MUZYKA, ZWIERZETA, SZTUKA, GAMING, FOTOGRAFIA...
25   </exploration_leads>
26

```

```

27 <tools>
28   <tool name="ask_a_question_with_answer_suggestions"/>
29   <tool name="end_conversation"/>
30   <tool name="flag_inappropriate_request"/>
31 </tools>
32 </system>

```

Wyniki optymalizacji. Iteracyjne doskonalenie promptów przyniosło wymierne rezultaty:

- Liczba pytań w wywiadzie: 10 → 30 (3× więcej danych o odbiorcy),
- Obszary eksploracji: 6 → 16+ kategorii życia,
- Sukces tool calls: ~60% → ~95% (dzięki auto-retry),
- Trafność pierwszej rekomendacji: wzrost do 64% (First Hit Accuracy).

Kluczowym wnioskiem z procesu prompt engineeringu jest znaczenie **konkretnych przykładów** i **explicite zakazów** w instrukcjach. Model lepiej rozumie „NIE pytaj o budżet” niż „skup się na zainteresowaniach”. Podobnie, podanie 2–3 przykładowych rozmów w prompcie drastycznie poprawia jakość generowanych pytań.

12.5 Równoległa Agregacja Ofert

Ze względu na limity API i czas odpowiedzi serwisów e-commerce, zaimplementowano mechanizm równoległego pobierania danych. **Fetch Microservice** może być uruchamiany w wielu instancjach, z których każda obsługuje innego dostawcę (OLX, Allegro, Amazon, eBay, Okazje). Wyniki są asynchronicznie przesyłane do głównego strumienia zdarzeń, co pozwala na prezentowanie pierwszych ofert użytkownikowi bez czekania na zakończenie przeszukiwania wszystkich źródeł.

Zaimplementowany system został poddany weryfikacji poprzez szereg testów, których przebieg i wyniki opisano poniżej.

13 Testy

13.1 Planowanie testów

Strategia testowania opierała się głównie na testach manualnych pełnego przepływu użytkownika, uzupełnionych o wybrane testy jednostkowe w obszarach najbardziej wrażliwych. Plan obejmował:

- **Testy automatyczne (zakres celowo wąski):** kilka testów jednostkowych w Stalking Microservice (np. `StalkingAnalyzeHandler`, `BrightDataService`) oraz podstawowe szkielety e2e/kontrolerów generowane przez NestJS w pozostałych mikroservisach. Celem było wychwycenie regresji w logice OSINT i poprawności konfiguracji datasetów; reszta pokrywana manualnie.
- **Testy manualne end-to-end (główny ciężar):**

- Pełny scenariusz wyszukiwania: logowanie, formularz (okazja, budżet, linki), wywiad z chatbotem, oczekiwanie na wyniki (SSE), przegląd i oznaczanie ulubionych, zapis historii.
 - Scenariusze alternatywne: brak linków społecznościowych, częściowa dostępność źródeł ofert (świadome wyłączanie jednego fetchera), przerwanie/ponowienie rozmowy, brak wyników z jednego sklepu.
 - Refinement: wybór kilku produktów, uruchomienie kolejnej rundy, weryfikacja, że nowe oferty dochodzą z poprawnym oznaczeniem rundy.
 - Panel admina i feedback: dodanie opinii (ocena, komentarz, opcjonalne zdjęcie), podgląd agregatów w panelu admina, przegląd opinii dla sesji.
 - UI/UX: responsywność (desktop/mobile), poprawność komunikatów o postępie („to może potrwać kilka minut”), zachowanie przy dłuższym oczekiwaniu na wyniki.
- **Testy danych i odporności:** ręczne sprawdzenie, jak system reaguje na „trudne” dane wejściowe (np. nietypowe linki, brak profili, długie odpowiedzi w czacie) oraz celowe wyłączanie pojedynczych usług (fetch Allegro) w pilotażu, aby ocenić odporność na częściowe awarie.

Testy manualne wykonywano iteracyjnie przy każdej większej zmianie (flow użytkownika, integracje fetch/reranking, UI), a zebrane uwagi wprost zasilały backlog poprawek (np. skrócenie wywiadu, dopisanie komunikatów o długim czasie wyszukiwania, balans źródeł ofert). Automatyzacja została ograniczona do najważniejszych fragmentów logiki, a główny nacisk położono na ręczną weryfikację doświadczenia użytkownika i komunikatów w sytuacjach brzegowych.

13.2 Testy z użytkownikami

Ważnym elementem weryfikacji jakości systemu były beta-testy z udziałem rzeczywistych użytkowników. Zebrano opinie dotyczące trafności rekomendacji (oceny gwiazdkowe, komentarze tekstowe) oraz wrażeń z korzystania z interfejsu (czas oczekiwania, przejrzystość, intuicyjność). Dane te zostały wykorzystane zarówno w sekcji wyników, jak i przy planowaniu dalszego rozwoju (np. dodanie pamięci negatywnej, lepsze dopytywanie o budżet).

Scenariusz testu z użytkownikami

Każdy uczestnik badania przechodził przez ten sam, ustandaryzowany scenariusz testowy:

1. **Wybór kontekstu:** Uczestnik wybierał realną osobę, dla której w najbliższym czasie planował kupić prezent (np. partner, przyjaciel, członek rodziny), określał okazję (urodziny, święta, rocznica itp.) oraz przybliżony budżet.
2. **Konfiguracja wyszukiwania:** Uczestnik wypełniał formularz startowy w aplikacji, podając — jeśli posiadał — linki do profili społecznościowych tej osoby oraz ustawiając budżet zgodnie z założeniami testu.
3. **Wywiad z chatbotem:** Uczestnik odpowiadał na kolejne pytania chatu, starając się udzielać pełnych odpowiedzi. Celem było doprowadzenie wywiadu do naturalnego zakończenia (bez świadomego przerywania konwersacji).

4. **Analiza rekomendacji:** Po zakończeniu wywiadu uczestnik przeglądał listę proponowanych produktów, porównując je z własną wiedzą o obdarowywanym (styl, zainteresowania, posiadane już przedmioty).
5. **Ocena całego procesu:** Uczestnik oceniał ogólne doświadczenie z systemem (czas oczekiwania, przejrzystość, intuicyjność, trafność rekomendacji) w formie oceny gwiazdkowej oraz krótkiego komentarza.
6. **Ocena pojedynczych produktów (opcjonalnie):** Uczestnik mógł wskazać wybrane produkty i wystawić im osobne oceny (np. które propozycje były szczególnie trafne, a które całkowicie nietrafne), co pozwalało lepiej zrozumieć jakość selekcji.

Jednym z głównych ryzyk, które miały wychwycić testy z użytkownikami, była odporność systemu na częściowe awarie integracji z serwisami e-commerce. W kilku sesjach celowo wyłączono jedną z instancji `Fetch Microservice` (np. odpowiedzialną za Allegro), aby sprawdzić, czy użytkownik nadal otrzyma sensowny zestaw propozycji z pozostałych źródeł i czy interfejs poprawnie komunikuje ograniczoną dostępność wyników.

W ramach testów szczególną uwagę zwrócono na scenariusz użytkownika szukającego prezentu dla fotografa-amatora. System zaproponował m.in. analogowy aparat z OLX, zestaw akcesoriów do aparatu oraz kurs fotografii online, a użytkownik ocenił całą sesję na 5/5 gwiazdek, podkreślając, że sam nie wpadłby na część tych pomysłów.

Pozytywne wyniki testów pozwoliły na przeprowadzenie badań z użytkownikami końcowymi, których rezultaty przedstawiono w ostatniej sekcji.

14 Wyniki i analiza badań

System został poddany testom w środowisku produkcyjnym (pilotaż, listopad 2025). W analizie wzięto pod uwagę 36 pełnych sesji zakupowych.

14.1 Efektywność czasowa

Średni czas potrzebny na znalezienie prezentu został zredukowany z rynkowej średniej 15 godzin (rocznie)¹ do **7 minut i 44 sekund** na sesję w AI Present Finder. W tym czasie system przeprowadza wywiad z użytkownikiem, analizuje profil społecznościowy i przeszukuje kilka platform e-commerce.

14.2 Jakość rekomendacji

- **Trafność (First Hit Accuracy):** 64% użytkowników uznało pierwszą propozycję za trafną.
- **Redukcja szumu:** Moduł rerankingu odrzuca średnio 40% ofert jako nietrafne lub niskiej jakości (np. części zamiennie zamiast produktów).
- **Satysfakcja użytkowników:** Średnia ocena 3.5/5.0. Użytkownicy docenili trafność pomysłów, ale wskazywali na problem polecania rzeczy, które obdarowywany już posiada (brak "pamięci negatywnej" w MVP).

¹Por. badania konsumenckie opisane w raporcie projektu AI Present Finder, oparte na danych z Consumer Reports [1].

14.3 Przykłady rerankingu AI (Explainable AI)

Moduł rerankingu wykorzystuje model Gemini 2.5 Flash Lite do oceny każdego produktu w skali 1–10 wraz z uzasadnieniem. Na podstawie danych z produkcji (2512 produktów, 2380 z oceną) przeanalizowano rozkład ocen (Tabela 4):

Zakres ocen	Liczba produktów	Procent
Ocena 1–3 (odfiltrowane)	391	16%
Ocena 4–5 (słabe dopasowanie)	267	11%
Ocena 6–7 (dobre dopasowanie)	779	33%
Ocena 8–9 (bardzo dobre)	899	38%
Ocena 10 (idealne)	44	2%
Średnia ocena	6.38/10	

Tabela 4: Rozkład ocen rerankingu AI (dane produkcyjne, grudzień 2025)

Produkty z oceną 10 (idealne dopasowanie, Tabela 5)

Tytuł	Cena	Ocena	Uzasadnienie AI
Herman Miller Aeron C full pakiet	3 800 zł	10	Najwyższej klasy fotel biurowy, znany z niezrównanej ergonomii i komfortu, idealnie wpisuje się w potrzeby osoby ceniącej wygodę siedzenia.
Fotel gamingowy Diablo X-Horn	500 zł	10	Wygodny i stylowy fotel gamingowy, zapewniający najwyższy komfort i funkcjonalność. Idealnie pasuje do potrzeb odbiorcy szukającego ergonomicznego fotela.
Fotel Secret Lab jak NOWY + akcesoria	1 599 zł	10	Fotel gamingowy Secret Lab jak nowy, z dodatkowymi akcesoriami i w świetnej cenie, jest idealnym prezentem.

Tabela 5: Przykłady produktów z oceną 10 (idealne dopasowanie)

Produkty z oceną 1 (odfiltrowane jako niepasujące, Tabela 6)

Tytuł	Cena	Ocena	Uzasadnienie AI
Duża kuchenka dla dzieci mega zestaw	189 zł	1	Duża kuchenka dla dzieci jest całkowicie nieodpowiednia dla odbiorcy w wieku 18–25 lat i nie ma związku z jego zainteresowaniami.
Koci Domek Gabi z figurkami	249 zł	1	Produkt jest zabawką przeznaczoną dla małych dzieci, co całkowicie mija się z zainteresowaniami i wiekiem odbiorcy.
Kuchnia drewniana IKEA dla dzieci	120 zł	1	Zabawka przeznaczona dla dzieci 1–3 lata, całkowicie nieodpowiednia dla dorosłego odbiorcy.

Tabela 6: Przykłady produktów z oceną 1 (odfiltrowane jako niepasujące)

Reranking skutecznie eliminuje produkty nietrafione kontekstowo (zabawki dla dzieci przy szukaniu prezentów dla dorosłych) i przyznaje wysokie oceny produktom dokładnie odpowiadającym profilowi odbiorcy. System odfiltrował 27% produktów (oceny 1–3) jako niepasujące.

14.4 Feedback użytkowników

W ramach testów pilotażowych zebrano 11 opinii od użytkowników. Poniżej przedstawiono reprezentatywne cytaty:

Opinie pozytywne

- *"Trafiony w sedno jak na tak niedługą i przyjemną rozmowę."* (Ocena: 5/5)
- *"Szybko, sprawnie, widać że system próbuje zrozumieć kontekst. Propozycje były sensowne i w budżecie."* (Ocena: 4/5)
- *"Świetny pomysł na aplikację! Wyniki były zaskakująco trafne."* (Ocena: 4/5)

Opinie krytyczne i sugestie

- *"Większość prezentów miała wspólne cechy z prezentami które były kupione w przeszłości — system polecał rzeczy, które osoba już ma."* (Ocena: 3/5) — wskazuje na potrzebę "pamięci negatywnej".
- *"OLX przytłaczające, brak wyników z Allegro. Rozmowa była zbyt długa."* (Ocena: 2/5) — sugestia balansowania źródeł (brak wyników wynikał z użycia środowiska Allegro Sandbox) i skrócenia wywiadu.

Główne wnioski z feedbacku:

1. 64% użytkowników uznało pierwszy wynik za trafny
2. Najczęstszy problem: polecenie przedmiotów już posiadanych przez obdarowywanego
3. Użytkownicy docenili szybkość działania i naturalność konwersacji

14.5 Stabilność

System osiągnął 100% dostępności (uptime) podczas testów, przy minimalnym zużyciu zasobów (<800MB RAM dla całego klastra), co potwierdza poprawność założeń architektonicznych.

15 Podsumowanie projektu

AI Present Finder zrealizował cel postawiony na początku przedsięwzięcia: połączył analizę publicznych danych (OSINT), konwersacyjny wywiad z użytkownikiem oraz agregację ofert z wielu platform e-commerce w spójny, działający system rekomendacji prezentów. Z perspektywy użytkownika końcowego aplikacja sprowadza się do prostego procesu: podaj

kontekst (okazja, osoba, budżet), porozmawiaj chwilę z AI, a następnie przejrzyj konkretną listę propozycji, które „mają sens” dla danej osoby.

Z perspektywy architektury udało się zrealizować pełny, produkcyjny układ mikroservisów: od bramy RestAPI (NestJS, JWT, Google OAuth), przez Stalking Microservice (BrightData, OSINT), Chat Microservice (LLM + tool calling), Gift Ideas i Fetch (równoległe pobieranie ofert z wielu źródeł), aż po Reranking Microservice oparty na modelu Gemini 2.5 z wyjaśnialnym scoringiem. Komunikacja asynchroniczna (RabbitMQ) i streaming SSE pozwoliły utrzymać dobrą ergonomię pracy użytkownika mimo wieloetapowego procesu „pod spodem”.

Z punktu widzenia wyników pilotażu system spełnił najważniejsze założenia: średni czas znalezienia prezentu skrócono z rzędu kilkunastu godzin rocznie do pojedynczej sesji trwającej około 7 minut i 44 sekund, przy zachowaniu wysokiej trafności pierwszej rekomendacji (64%) oraz skutecznym odfiltrowaniu około 40% szumu ofertowego przez moduł rerankingu. Jednocześnie utrzymano wysoką stabilność techniczną — w trakcie testów nie odnotowano przerw w dostępności aplikacji, a zużycie zasobów pozostało na poziomie akceptowalnym dla środowiska chmurowego.

Zespół przeszedł pełny cykl życia projektu: od POC i weryfikacji ryzyk technologicznych, przez fazę projektowania (diagramy C4, BPMN, model danych), implementację i testy, aż po pilotaż produkcyjny z rzeczywistymi użytkownikami. Ostateczna dokumentacja (niniejszy dokument, raport, diagramy i opisy flow użytkownika) odzwierciedla nie tylko stan końcowy systemu, ale też przyjętą filozofię projektowania: łączenie pragmatycznej inżynierii z czytelną komunikacją „co” i „dlaczego” zostało zrobione.

16 Wnioski i kierunki dalszego rozwoju

Wdrożenie AI Present Finder pozwoliło w praktyce zweryfikować założenia tzw. *Agentic Commerce*: dużą część procesu decyzyjnego przy wyborze prezentu można przekazać autonomicznemu agentowi AI, który sam zbiera kontekst, przeszukuje rynek i selekcionuje oferty. Analiza wyników, feedbacku użytkowników i doświadczeń zespołu prowadzi do kilku nadrzędnych wniosków:

- **Połączenie OSINT + wywiad AI działa.** Użytkownicy docenili fakt, że system „zna kontekst” — łączenie publicznych profili z rozmową pozwala generować rekomendacje bliższe realnym zainteresowaniom obdarowywanej osoby niż klasyczne filtry typu kategoria/cena.
- **Reranking z uzasadnieniami ma dużą wartość dodaną.** Moduł oceniający produkty z komentarzem AI skutecznie usuwa oferty przypadkowe, skracając listę do propozycji, które faktycznie da się rozważyć. Jednocześnie taki sposób działania jest łatwy do audytu i dalszego strojenia.
- **Streaming i asynchroniczność są konieczne przy długich procesach.** Pełne wyszukiwanie (OSINT, kilkaset produktów z wielu sklepów, reranking) może trwać kilka minut, ale dzięki SSE użytkownik na każdym etapie widzi, że system pracuje i stopniowo dostaje wyniki. To podejście okazało się niezbędne dla utrzymania zaangażowania.
- **Największe ograniczenia leżą po stronie danych.** Główne problemy zgłaszane w badaniu (polecanie rzeczy, które dana osoba już posiada, oraz bias w stronę jed-

nego źródła ofert) wynikają z brakującej „pamięci negatywnej” i nierównomiernej dostępności danych z poszczególnych platform.

Na tej podstawie można wskazać naturalne kierunki dalszego rozwoju systemu:

- **Pamięć negatywna i lepsze modelowanie historii.** Dodanie mechanizmów zapamiętywania produktów już posiadanych oraz prezentów odrzuconych przez użytkownika pozwoli ograniczyć polecenie „tych samych rzeczy” i jeszcze lepiej wykorzystać zebrany feedback.
- **Balansowanie i rozszerzanie źródeł ofert.** Z punktu widzenia użytkownika ważne jest, aby lista prezentów była różnorodna zarówno pod względem rodzaju produktów, jak i źródeł (nowe vs. używane, różne platformy). W praktyce oznacza to dalszą pracę nad integracjami oraz logiką ważenia źródeł.
- **Optymalizacja czasu reakcji i kosztów.** Choć obecny czas do wyników jest akceptowalny w kontekście pilotażu, celem kolejnego etapu jest dalsze skrócenie opóźnień (docelowo do kilku minut lub mniej) przy utrzymaniu lub obniżeniu kosztów operacyjnych, m.in. poprzez lepsze wykorzystanie streamingu, cache i odpowiednio dobrane modele.
- **Wyszukiwanie wektorowe i rekomendacje oparte na feedbacku.** Dodanie wyszukiwania semantycznego (embeddingi) oraz mechanizmów uczących się z ocen użytkowników pozwoliłoby wychodzić poza prostą filtrację ofert i stopniowo „dostrajać” system do preferencji społeczności używającej aplikacji.
- **Wywiad adaptacyjny i personalizacja interfejsu.** Zebrane opinie pokazują, że część użytkowników chciałaby krótszych lub bardziej „na temat” rozmów. Naturalnym kierunkiem jest dalsze upraszczanie dialogu oraz dopasowanie interfejsu do mniej technicznych użytkowników.
- **Eksploracja modelu biznesowego.** Przy zachowanych parametrach jakości i stabilności AI Present Finder może stanowić bazę pod komercyjne rozwiązanie (np. model afiliacyjny, integracje dla sklepów, wersja Pro dla agencji marketingowych). Obecne MVP pokazuje, że technicznie jest to wykonalne.

Podsumowując, projekt AI Present Finder potwierdził, że połączenie OSINT, konwersacyjnego AI i świadomie zaprojektowanej architektury mikroservisowej pozwala realnie poprawić doświadczenie kupowania prezentów. Jednocześnie wskazał, że kluczem do dalszego wzrostu jakości nie są już tylko „mocniejsze modele”, lecz przede wszystkim bogatsze dane, lepsze wykorzystanie feedbacku użytkowników i dopracowanie doświadczenia końcowego odbiorcy.

Literatura

- [1] Consumer Reports. *Holiday Shopping Survey: Time Spent and Gift Returns*, 2010. Dostępne w raporcie głównym projektu AI Present Finder.